دانشگاه اصفهان
دانشکده مهندسی کامپیوتر

پایگاه داده پیشرفته - مقطع کارشناسی ارشد

گزارش پروژه پایانی پایگاه داده پیشرفته

پلتفرم هوشمند تحلیل داده‌های شرکت Uber

حسین مستاجران گورتانی - ۴۰۴۳۶۱۴۰۳۳

لینک ریپازیتوری:
https://github.com/HosseinMst81/uber-data-platform/

بهمن ۱۴۰۴

چکیده

این پروژه پایانی درس پایگاه داده پیشرفته، طراحی و پیاده‌سازی یک پلتفرم هوشمند تحلیل داده‌های تراکنش‌های شرکت Uber در سال ۲۰۲۴ را با استفاده از معماری Medallion (Bronze، Silver، Gold) به انجام رساند. ابتدا داده‌های خام با ۱۵۰,۰۰۰ رکورد بدون هیچ تغییری در لایه Bronze ذخیره شد. سپس در لایه Silver با انجام عملیاتETL ، مهندسی ویژگی (اضافه کردن datetime و ستون‌های زمانی)، یکپارچه‌سازی دلایل لغو سفر در یک ستون واحد و مدیریت هوشمند مقادیر Null (به‌ویژه پر کردن میانگین امتیازها در سفرهای تکمیل‌شده) داده‌ها تمیز و آماده تحلیل شد. در لایه Gold ، جدول نهایی با کلید اصلی، محدودیت‌های CHECK روی امتیازها و محاسبه متریک کلیدی revenue_per_km ساخته شد تا داده‌ها برای گزارش‌گیری و داشبورد بهینه باشند.

یک API ساده CRUD روی لایه Gold پیاده‌سازی شد تا امکان ایجاد، مشاهده (با فیلتر customer_id) ، ویرایش وضعیت و حذف سفرها فراهم شود. داشبورد تعاملی با شاخص‌های کلیدی عملکرد (تعداد سفرها، درآمد کل، نرخ موفقیت، توزیع لغوها و ...) و نمودارهای متنوع (Pie، Bar، Line) با قابلیت فیلتر زمانی و نوع خودرو طراحی گردید. در بخش دستیار هوشمند Text-to-SQL ، با استفاده از مدل Llama از طریق OpenRouter و پرامپت مهندسی‌شده شامل schema جدول، سوالات طبیعی به کوئری SELECT امن تبدیل می‌شوند. در نهایت با ایجاد ایندکس‌های مناسب و مقایسه زمان اجرا با EXPLAIN ANALYZE، بهبود عملکرد دیتابیس سنجیده شد و جستجوی معنایی روی دلایل لغو با Chroma DB پیاده‌سازی گردید.

این سیستم ترکیبی از مفاهیم پایگاه داده پیشرفته، مهندسی داده و هوش مصنوعی را در یک پلتفرم کاربردی و کامل به نمایش گذاشت.

فهرست مطالب

صفحه

مرحله اول: آشنایی با دیتاست..... ۷
۱-۲- مرحله دوم: طراحی معماری پایه بر اساس Medallion Architecture لایه Bronze..... ۷
۲-۲- مرحله سوم: طراحی معماری پایه بر اساس Medallion Architecture لایه Silver..... ۸
۳-۲- مرحله چهارم: طراحی معماری پایه بر اساس Architecture Medallion لایه Gold..... ۹
۴-۲- مرحله پنجم: پیاده سازی CRUD API..... ۹
۵-۲- مرحله ششم: داشبور تحلیلی و تعاملی..... ۹
۶-۲- مرحله هفتم: دستیار هوشمند Text to SQL..... ۱۰
۷-۲- مرحله هشتم: بهینه سازی و ایندکس گذاری..... ۱۰
۸-۲- مرحله نهم: پیاده سازی Vector Database و جستجوی معنایی (Semantic Search)..... ۱۱
پیاده‌سازی پروژه..... ۱۲
گزارش فاز اول: تحلیل و شناسایی مجموعه داده(Data Analysis)..... ۱۳
۱-۳-۲- توصیف کلی (General Description)..... ۱۳
۲-۳-۲- تحلیل ستون‌ها و روابط بین آن‌ها..... ۱۴
گزارش فاز دوم: Medalion Architecture لایه Bronze..... ۱۸
گزارش فاز سوم: Medalion Architecture لایه Silver..... ۲۳
۹-۲- راه‌اندازی اولیه تراکنش..... ۲۳
2-1- اقدام اول مهندسی ویژگی (Feature Engineering)..... ۲۴
۱-۳-۲- پاکسازی داده‌های قبلی و درج اولیه از برنز..... ۲۵
2-2- اقدام دوم: یکپارچه‌سازی دلایل لغو سفر..... ۲۸
۳-۲- اقدام سوم: مدیریت مقادیرNull..... ۲۸
۴-۲- اقدام چهارم: تحلیل و مدیریت امتیازها..... ۲۹
۵-۲- پایان تراکنش و لاگ نهایی..... ۳۱
2-6- بررسی تعداد رکوردها و مدیریت داده‌های تکراری..... ۳۶
گزارش فاز چهارم: Medalion Architecture لایه Gold..... ۳۸
گزارش فاز پنجم: پیاده‌سازی CRUD API..... ۴۲
۱-۲- ساختار تعریف شده Backend:..... ۴۲
2-2- عملیات CRUD..... ۴۴
2-3-1- CREATE - Post API..... ۴۵

۲-۳-۲- READ - GET API ۴۶
۲-۳-۳- UPDATE - PATCH API ۴۸
۲-۳-۴- DELETE API ۴۹

گزارش فاز ششم : داشبورد تحلیلی و تعاملی ۵۰

۲-۱- طراحی معماری سمت بک‌اند برای داشبورد ۵۰
۲-۲- مدیریت ایمن مقادیر عددی ۵۱
۲-۳- شاخص‌های کلیدی عملکرد(KPI) ۵۲
۲-۴- نمودارها(Charts) ۵۴
۲-۳-۱- نمودار دایره‌ای توزیع دلایل لغو ۵۴
۲-۳-۲- نمودار توزیع روش‌های پرداخت ۵۶
۲-۳-۳- نمودار مقایسه نوع خودرو ۵۷
۲-۳-۴- نمودار ساعات پرتردد ۵۸
۲-۳-۵- نمودار تحلیل روزهای هفته ۵۹

گزارش فاز هفتم: دستیار هوشمند Text to SQL ۶۱

۲-۱- پیاده‌سازی بک‌اند ۶۱

گزارش فاز هشتم: بهینه‌سازی و ایندکس‌گذاری ۶۵

۲-۱- انتخاب کوئری سنگین ۶۵
۲-۲- اجرای کوئری قبل از ایندکس‌گذاری ۶۶
۲-۳- ایجاد ایندکس‌های مناسب ۶۸
۲-۴- خروجی کوئری با استفاده از ایندکس ۶۹
۲-۵- مقایسه ۷۰

منابع و مراجع ۷۲

پیوست‌ها: ۷۴

فهرست اشکال

صفحه

شکل ۱ دیاگرام موجودیت‌ها و روابط (ER Diagram) ۱۷
شکل ۲ راه‌اندازی اولیه PostgreSQL از طریق Shell psql ۱۸
شکل ۳ ساختن اسکیمای Bronze و ساخت جدول لایه اول Bronze Layer ۱۹
شکل ۴ ارور وجود null رشته‌ای در فایل csv ۲۰
شکل ۵ کد وارد کردن داده‌های خام از فایل csv به داخل bronze schema و جدول raw_dataset ۲۱
شکل ۶ چک کردن اتصال به PostgreSQL ۲۱
شکل ۷ اسکیماها و جداول ساخته شده در PostgreSQL DB ۲۲
شکل ۸ لایه Bronze و raw_dataset در PostgreSQL DB ۲۲
شکل ۹ اجرای موفق کد silver_etl و لایه silver ۳۱
شکل ۱۰ ساختار جدول Bronze ۳۱
شکل ۱۱ ساختار جدول Silver ۳۲
شکل ۱۲ داده‌های لایه برنز قبل از تغییرات در pgAdmin4 ۳۲
شکل ۱۳ داده‌ها در لایه Silver بعد از تغییرات در pgAdmin4 ۳۳
شکل ۱۴ مقایسه null value ها در دو لایه نقره و برنز و درصد بهبود ۳۳
شکل ۱۵ جدول Silver و مدیریت مقادیر Null ۳۴
شکل ۱۶ عدم وجود Null در ستون جدید unified_cancellation_reason ۳۴
شکل ۱۷ دلایل لغو سفر و تعداد هرکدام به صورت یکپارچه ۳۵
شکل ۱۸ میانگین امتیازها با توجه به نوع خودرو ۳۶
شکل ۱۹ بررسی وجود booking_id تکراری ۳۷
شکل ۲۰ مقایسه تعداد رکوردهای جدول Silver وBronze ۳۷
شکل ۲۱ خروجی موفق عملیات Gold ۳۹
شکل ۲۲ ساختار و اسکیمای جدول Gold ۳۹
شکل ۲۳ میانگین درامد در هر کلیومتر به تفکیک نوع وسیله نقلیه ۴۰
شکل ۲۴ جدول Gold ۴۱
شکل ۲۵ Create - ساخت یک سفر جدید ۴۵
شکل ۲۶ READ - خواندن یک رکورد با customer_id مشخص ۴۶
شکل ۲۷ READ-خواندن همه رکوردها یا اعمال فیلتر با کمک limit و offset ۴۷
شکل ۲۸ READ - خواندن همه رکوردها بدون هیچ فیلتری ۴۷
شکل ۲۹ UPDATE - بروزرسانی وضعیت یک سفر از طریق booking_id ۴۸
شکل ۳۰ DELETE - حذف یک سفر ۴۹

شکل ۳۱ کد تابع safeNumber برای مدیریت مقادیر عددی ۵۲
شکل ۳۲ نمایش KPI ها در داشبورد ۵۳
شکل ۳۳ نمایش KPI ها با فیلتر نوع خودرو ۵۳
شکل ۳۴ نمایش KPI ها با فیلتر زمانی ۵۳
شکل ۳۵ کد مربوط به سفرهای لغو شده ۵۴
شکل ۳۶ نمودار دلایل لغو سفر ۵۵
شکل ۳۷ نمودار دایره‌ای روش‌های پرداخت ۵۶
شکل ۳۸ نمودار میله‌ای مقایسه انواع خودرو ۵۸
شکل ۳۹ نمودار خطی ساعات پرتردد ۵۹
شکل ۴۰ نمودار میله‌ای تعداد سفرها براساس روز هفته ۶۰
شکل ۴۱ توصیف GOLD_SCHEMA برای AI درکنترلر ۶۱
شکل ۴۲ تعریف محدودیت‌ها و اعتبارسنجی به صورتPrompt Engineering ۶۲
شکل ۴۳ دستیارهوشمند Text to SQL ۶۳
شکل ۴۴ سوال پیچیده در دستیار هوشمند ۶۴
شکل ۴۵ سوال نامربوط در دستیار هوشمند ۶۴
شکل ۴۶ اجرای کوئری سنگین بدون ایندکس‌گذاری ۶۶
شکل ۴۷ خروجی کوئری سنگین بدون ایندکس‌گذاری ۶۷
شکل ۴۸ خروجی کوئری سنگین با استفاده از ایندکس ما ۶۹

فهرست جداول

صفحه

جدول1 معرفی ستون ها و نوع داده‌ها (Featuring & Data Types)..... ۱۶
جدول2 مقایسه سه لایه معماری مدالیون..... ۴۱
جدول3 مقایسه استفاده از ایندکس و عدم استفاده از ایندکس مناسب..... ۷۰

مرحله اول: آشنایی با دیتاست

تحلیل زیرساختی و درک ماهیت داده‌ها: در گام نخست، تمرکز اصلی بر شناخت دقیق دیتاست واقعی تراکنش‌های شرکت Uber است که شامل بیش از ۱۴۸ هزار رکورد است.

قبل از هرگونه طراحی پایگاه داده، لازم است تا ساختار داده ها را با دقت بررسی کنید. لطفا موارد زیر را انجام دهید:

توصیف کلی: درک کلی خود از دیتاست، مقیاس آن و حوزه کاربرد را بیان کنید.

معرفی ستون ها (Features): برای هر یک از ستون های موجود، دقیقا مشخص کنید که چه اطلاعاتی نگهداری می‌کند و نوع داده آن‌ها چیست؟

ارتباطات منطقی: رابطه بین ستون ها را بر اساس منطق کسب و کار توضیح دهید.

در پایان این مرحله شما باید درک کاملی از ماهیت هر ستون و نحوه پر شدن داده ها در دیتاست داشته باشید

۱-۲- مرحله دوم: طراحی معماری پایه بر اساس Medallion Architecture لایه Bronze

لایه Bronze یا Raw Layer جایی است که داده ها دقیقا همانطور که از منبع آمده اند، بدون هیچگونه تغییر ذخیره می شوند.

در دیتابیس PostgreSQL یک Schema جدید به نام bronze ایجاد کنید. یک جدول به نام raw_dataset بسازید.

ستون های جدول را بر اساس تحلیلی که در مرحله اول انجام داده اید و با Data Type مناسب تعریف کنید و دیتاست اصلی را بدون تغییر، در جدول Import کنید.

هدف این مرحله Raw Ingestion است و صرفا حفظ اصالت داده ها مدنظر است.

۲-۲- مرحله سوم: طراحی معماری پایه بر اساس Medallion Architecture لایه Silver

در این فاز، باید عملیات ETL را انجام دهید تا داده ها برای تحلیل آماده شوند. خروجی این مرحله باید در جدولی با نام silver.cleaned_dataset ذخیره شود.

اقدام اول: مهندسی ویژگی (Feature Engineering)

بررسی کنید آیا فرمت تاریخ و ساعت استاندارد است؟ با نظر خودتان می توانید یک ستون جدید برای استخراج جزئیات زمان یا تاریخ اضافه کنید. دلیل خود را برای این کار بیان کنید. با بررسی کامل دیتاست، تحلیل کنید که آیا با اضافه کردن ستون های جدید، می توان ستون های قدیمی را حذف کرد؟

اقدام دوم: یکپارچه سازی دالیل لغو سفر سه ستون مختلف شامل دالیل لغو توسط راننده، مشتری یا سفر ناقص وجود دارد. یک ستون جدید به نام Unified_cancellation_reason ایجاد کنید. با توجه به ستون Booking Status دلیل لغو صحیح را از یکی از سه ستون برداشته و در ستون جدید ذخیره کنید. اقدام سوم: مدیریت مقادیر Null بررسی کنید کدام ستون ها حجم زیادی از مقادیر Null دارند. آیا حذف این ردیف ها منطقی است؟ راهکار شما برای مدیریت این حجم Null چیست؟ اقدام چهارم: تحلیل و مدیریت امتیازها ستون های Driver Ratings و Customer Rating را بررسی کنید. Null بودن این مقادیر همیشه به معنی ناقص بودن داده ها نیست. برای وضعیت های مختلف بررسی کنید که Null بودن امتیاز چه معنایی دارد؟ در سفرهای Completetd بررسی کنید رفتار مشتریان و رانندگان برای ثبت امتیاز چگونه بوده است؟ آیا با داده های موجود می توانیم برای Customer Rating و سفرهای تکمیل شده چاره ای بیندیشیم تا از حجم Null بودن کم کنیم؟ دقت کنید که از حذف ردیف ها به دلیل نبودن امتیاز خودداری کنید!

۲-۳- مرحله چهارم: طراحی معماری پایه بر اساس Medallion Architecture لایه Gold

لایه Gold، لایه نهایی و قابل اتکا برای تحلیل کسب و کار و تولید گزارشات است. داده ها در این الیه باید بهینه سازی شده، غنی و آماده خواندن سریع باشند. اقدام اول: طراحی Schema جدول نهایی خود را با نام gold.dataset ایجاد کنید. از انواع داده دقیق و بهینه استفاده کنید. اقدام دوم: محاسبه و غنی سازی داده ها یک ستون جدید محاسباتی اضافه کنید. به عنوان مثال reveneue_per_km که حاصل تقسیم Booking Value بر Ride Distance است. اقدام سوم: یکپارچگی داده این جدول باید دارای Primary Key باشد. از محدودیت CHECK برای اطمینان از صحت داده ها استفاده کنید. مثال اطمینان حاصل کنید که امتیازات داده شده بین ۰ تا ۵ است.

۲-۴- مرحله پنجم: پیاده سازی CRUD API

در این مرحله، باید پلی میان کاربر و دیتابیس ایجاد کنید. شما می توانید با هر زبانی یا فریم ورکی که به آن تسلط دارید، مانند Go، Django، FastAPI، Node JS و ... این پیاده سازی را انجام دهید. هدف اصلی این مرحله، انجام عملیات CRUD روی الیه Gold است. اندپوینت های زیر را پیاده سازی کنید: ساخت سفر جدید (Create) با تولید خودکار Booking ID، نمایش لیستی از سفرها (Read) با قابلیت فیلتر بر اساس Customer ID، تغییر وضعیت (Update) و حذف (Delete) یک رکورد سفر از دیتابیس. برای نمایش عملیات CRUD، از Streamlit استفاده کنید.

۲-۵- مرحله ششم: داشبور تحلیلی و تعاملی

در این مرحله با استفاده از Streamlit یا ابزارهای مشابه، یک داشبورد تعاملی بسازید که وضعیت کسب و کار را به تصویر بکشد [۱۳]. اقدام اول: شاخص های کلیدی عملکرد (KPIs) تعداد کل رزروها، تعداد کل رزروهای موفق، مجموع درآمد و نرخ موفقیت سفر [۱۳، ۱۴]. اقدام دوم: نمودارها نمودار Pie Chart برای نمایش درصد یا توزیع دالیل لغو سفر و روش های مختلف پرداخت؛ نمودار Bar Chart برای

مقایسه تعداد سفرهای انجام شده برای هر نوع خودرو و نمایش میانگین امتیاز؛ نمودار خطی برای نمایش ساعت های پر تردد و روزهای هفته بر اساس تعداد سفر. اقدام سوم: تعاملی بودن داشبورد باید دارای فیلترهای انتخاب بازه زمانی و انتخاب نوع خودرو باشد. با تغییر فیلترها، تمام شاخص ها و نمودارها باید به روز رسانی شوند.

۲-۶- مرحله هفتم: دستیار هوشمند Text to SQL

در این قسمت شما باید یک رابط کاربری چت بسازید که بتواند سواالت مدیران را به زبان انگلیسی دریافت کرده و پاسخ آن را از دیتابیس استخراج کند. معماری Workflow: ورودی سوال متنی کاربر به زبان انگلیسی است. تولید SQL توسط AI با دادن اطالعات ساختار جدول Gold به مدل زبانی انجام می‌شود. اعتبارسنجی و فیلتر: کد شما باید خروجی مدل را قبل از اجرا بررسی کند؛ دستورات مخرب مانند DROP، DELETE، UPDATE و INSERT باید بالک شوند. الگوهای خطرناک مانند SELECT * FROM بدون LIMIT باید بالک شوند (اگر کوئری بدون LIMIT بود، LIMIT 10 را به آن اضافه کنید). سیستم شما فقط باید به درخواست های SELECT پاسخ دهد.

۲-۷- مرحله هشتم: بهینه سازی و ایندکس گذاری

در این مرحله باید به صورت عملی تاثیر ایندکس گذاری بر بهبود عملکرد دیتابیس را بسنجید. اقدام اول: انتخاب و نوشتن کوئری سنگین یک کوئری SELECT بنویسید که شامل فیلترهای ترکیبی یا عملیات Aggregation باشد. اقدام دوم: قبل از ایجاد ایندکس کوئری را روی جدول Gold اجرا کنید و از دستور EXPLAIN ANALYZE استفاده کنید تا زمان اجرای واقعی و پلن اجرایی را مشاهده کنید اقدام سوم: ایندکس گذاری با توجه به شناخت از دیتاست، یک یا چند ایندکس مناسب (مانند Composite Index) بسازید. دلیل انتخاب ستون های ایندکس را به صورت تکنیکال توجیه کنید. مجددا همان کوئری دقیق را اجرا کرده و زمان اجرا را قبل و بعد از ایندکس گذاری مقایسه کنید

۸-۲- مرحله نهم: پیاده سازی Vector Database و جستجوی معنـایی (Semantic Search)

در این مرحله، شما باید یک دیتابیس برداری (Vector Database) را به پروژه اضـافه کنیـد تـا قابلیـت جستجوی معنایی روی دالیل لغو سفر ایجاد کنید. اقدام اول: انتخاب ابزار و طراحی دیتـابیس بـرداری از Chroma DB اسـتفاده کنیـد. سـتون Unified_cancellation_reason را بـه همـراه Booking ID در Collection در Chroma DB وارد کنید. اقدام دوم: پیاده سازی اندپوینت جستجو یک بخـش جسـتجو جدید بسازید که یک عبارت متنی درباره علت کنسل شدن سفرها از کاربر دریافت کند. این عبارت را در Chroma DB جستجو کنید تا ۵ مورد (Top_k = 5) که بیشترین شـباهت معنـایی را دارنـد برگردانـد. سیستم باید از شناسه های سفر (Booking ID) برای واکشی جزئیات کامل از جـدول gold.dataset در PostgreSQL استفاده کرده و نتیجه را به کاربر نمایش دهد.

پیاده‌سازی پروژه

گزارش فاز اول: تحلیل و شناسایی مجموعه داده (Data Analysis)

۱-۳-۲- توصیف کلی (General Description)

خب اول از همه فایل رو با Excel باز کردیم، چند سطر اول و آخر رو نگاه کردیم و بعد با Python و کتاب‌خانه pandas یک خلاصه سریع گرفتیم تا متوجه بشیم دقیقا چی و چه دیتاهایی داخل این دیتاست هست.

این دیتاست شامل ۱۴۸,۷۷۰ رکورد از تراکنش‌های واقعی شرکت Uber در سال ۲۰۲۴ است.

ما با یک سیستم کاملاً زنده و عملیاتی طرف هستیم. این دیتاست صرفاً چند تا عدد و رقم ساده نیست این داده‌ها همه چیزهایی رو که تو یک سفر Uber اتفاق می‌افته ثبت کردن: از لحظه‌ای که مشتری درخواست خودش رو داده، تا وقتی سفر تموم می‌شه یا لغو می‌شه. به همین دلیل هم مقیاس دیتا واقعا بزرگه (نزدیک ۱۵۰ هزار سفر)، پس کاملاً مناسب تحلیل‌های واقعی کسب‌وکار و عملیات های تحلیل و آنالیز داده هست.

حوزه کاربردش هم بسیار واضح هست: کمک به مدیر شرکت Uber که بفهمه کدوم ساعات شلوغ تر هست، مردم بیشتر با چه ماشین‌هایی سفر می‌کنند؟ چرا سفرها لغو می‌شوند؟ درآمد از کجا بدست آمده و...

کاربرد اصلی این داده‌ها برای ما در اینجا، این هست که بفهمیم کجاها سیستم داره ضرر میده مثلاً چرا سفرها لغو میشن؟ چون نرخ لغو سفر بالا هست! و کجاها بیشترین درآمد رو داره؟، عملکرد رانندگان و مشتری‌ها چطوری بوده؟ تا بتونیم الگوهای پنهان داده‌ها و سفر ها رو بیرون بکشیم و در نهایت به بهینه‌سازی درآمد بر اساس نوع وسیله نقلیه (از موتور و دوچرخه برقی تا ماشین‌های لوکس و..) و مسافت و بقیه پارامترها بتونیم برسیم.

۲-۳-۲- تحلیل ستون‌ها و روابط بین آن‌ها

حالا بریم سراغ ستون‌ها. من هر کدوم رو نگاه کردم و سعی کردم بفهمم چی نگه می‌دارن و از چه نوعی تشکیل شده‌اند. مثلا:

- Date: تاریخ سفر که قطعا باید از نوع DATE باشه چون تاریخ کامل مورد نیاز هست.
- Time: زمان دقیق، مثل 12:29:38 که باید از نوعTIME یا TIMESTAMP باشه. این ستون البته میتونه با تاریخ ترکیب بشه در آینده.
- Booking ID: شناسه منحصربه‌فرد هر رزرو مثل CNR5884300، که می‌تونه از نوع VARCHAR تعریف بشه و قاعدتا میتونه کلید اصلی باشه.
- Booking Status: وضعیت سفر، مانند Completed یا Incomplete که باز هم VARCHAR مناسب هست.
- Customer ID: شناسه مشتری، مثل بالا از نوع VARCHAR
- Vehicle Type: نوع وسیله، مثل eBike یا Auto ، ازنوع VARCHAR
- Cancelled Rides by Customer: لغو سفر توسط مشتری که یک فلگ هست و null یا INTEGER (عدد ۱) هست. این میتونه بعدا به بولین تبدیل بشه.
- Reason for cancelling by Customer: دلیل لغو توسط مشتری، که TEXT مناسبه
- Cancelled Rides by Driver: لغو سفر توسط راننده که یک فلگ هست و null یا INTEGER (عدد ۱) هست. این میتونه بعدا به بولین تبدیل بشه.
- Driver Cancellation Reason: دلیل لغو سفر توسط راننده، که TEXTعه.
- Incomplete Rides: تعداد سفرهای تکمیل نشده،که عدد ساده یا INTEGER
- Incomplete Rides Reason: دلیل سفر تکمیل نشده TEXT
- Booking Value: ارزش مالی که برای پول DECIMAL
- Ride Distance: مسافتDECIMAL

- Driver Ratings: امتیاز راننده که عدی بین ۰ تا ۵ است و DECIMAL هست.
- Customer Rating: امتیاز مشتری، مشابه قبلی.
- Payment Method: روش پرداخت که VARCHAR

این ستون‌ها پر از null هستند که منطقیه چون مثلا اگر سفر کامل بشه، دلایل لغو خالی می‌مونه.

در مورد ارتباطات، همانطور که مشخص هست، همه چیز رو یک رزرو با Booking ID تعریف می‌کنه. برای مثال Customer ID به رزروها وصل می‌شه، یعنی یک مشتری می‌تونه چند سفر داشته باشه و هرسفر مربوط به یک مشتری هست.

Booking Status تعیین می‌کنه کدوم دلایل لغو سفر پر بشوند اگر Completed باشه، بقیه null هستن.

Vehicle Type و Payment Method هم به رزرو وابسته‌ان و می‌تونن برای تحلیل گروهی استفاده بشوند. مسافت و Booking Value هم با هم مرتبط هستند و می‌شه ازشون درآمد حساب کرد. امتیازها فقط برای سفرهای موفق معنی دارند و null بودنشون، نشون‌دهنده این هست که طرف امتیازی نداده است. کلا، داده‌ها طوری هستن که می‌تونیم الگوهای زمانی (از Date و Time) یا رفتاری (مثلا از لغوها) رو استخراج کنیم.

برای اینکه بهتر نشون بدیم، یک دیاگرام ER ساده کشیدم که موجودیت‌ها و روابط رو نشون می‌ده. این صرفا بر اساس فهم خودم از دیتاست هست:

اگر وضعیت سفر "No Driver Found" باشه، عملاً ستون‌های قیمت و مسافت و امتیاز همگی خالی (null) می‌مونن چون اصلاً سفری شروع نشده که بخواد قیمتی داشته باشه.

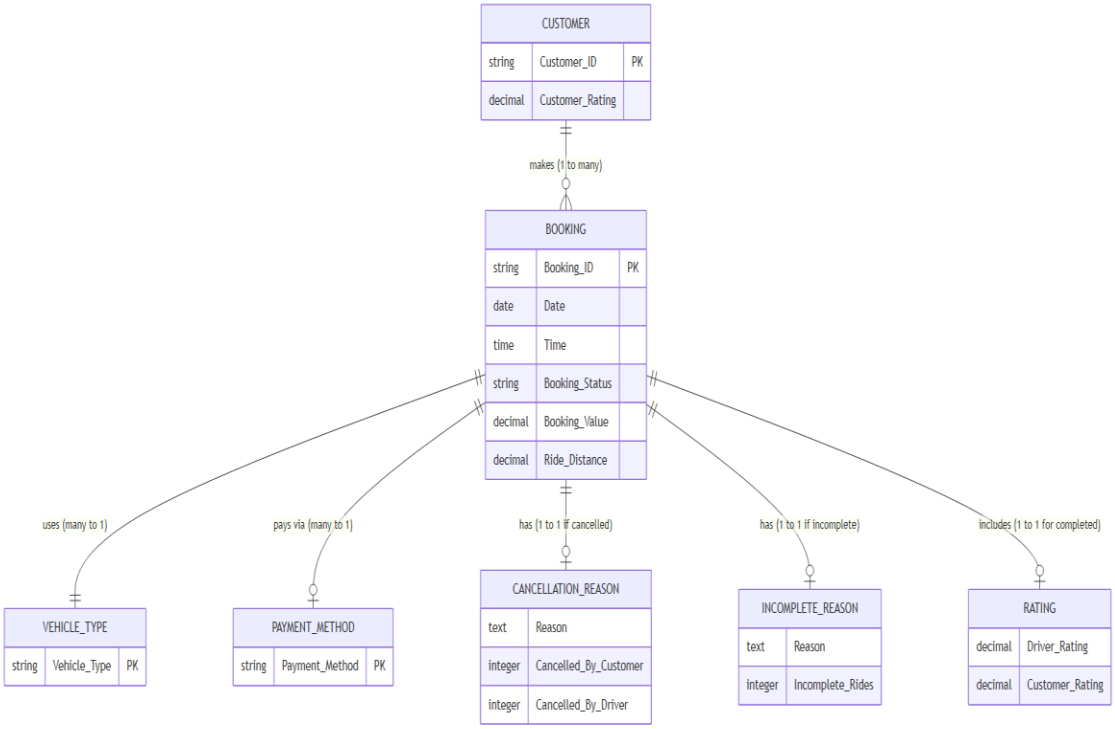
یک رابطه مستقیم بین "Vehicle Type" و "Booking Value" وجود داره؛ مثلا قطعا قیمت یک ماشین لوکس در یک مسیر مشخص باید از یک وسیله نقلیه ارزان بیشتر باشه.

در این بخش، ماهیت هر ستون و نوع داده‌ای پیشنهادی برای ذخیره‌سازی بهینه در PostgreSQL مشخص شده است:

جدول1 معرفی ستون ها و نوع داده‌ها (Featuring & Data Types)

نام ستون در دیتاست	شرح اطلاعات	نوع داده (Data Type)
Date	تاریخ ثبت درخواست سفر	DATE
Time	زمان دقیق ثبت درخواست	TIME
Booking ID	شناسه منحصربه‌فرد برای هر تراکنش (Primary Key)	VARCHAR(20)
Booking Status	وضعیت فعلی سفر (Completed, Cancelled, etc.)	VARCHAR(50)
Customer ID	شناسه منحصربه‌فرد مشتری	VARCHAR(20)
Vehicle Type	نوع وسیله نقلیه(eBike, Go Sedan, Auto, etc.)	VARCHAR(30)
Cancelled Rides by Customer	نشانگر لغو توسط مشتری (معمولاً ۱ یا null)	BOOLEAN یا SMALLINT
Reason for cancelling by Customer	علت لغو سفر از سمت مشتری	TEXT
Cancelled Rides by Driver	نشانگر لغو توسط راننده (معمولاً ۱ یا null)	BOOLEAN یا SMALLINT
Driver Cancellation Reason	علت لغو سفر از سمت راننده	TEXT
Incomplete Rides	نشانگر سفرهایی که نیمه‌کاره مانده‌اند	BOOLEAN یا SMALLINT
Incomplete Rides Reason	علت اتمام نیمه‌کاره سفر (مانند نقص فنی)	TEXT
Booking Value	مبلغ کل تراکنش	DECIMAL(10, 2)
Ride Distance	مسافت طی شده (بر حسب واحد مسافت)	DECIMAL(8, 2)
Driver Ratings	امتیاز داده شده به راننده (۰ تا ۵)	DECIMAL(2, 1)
Customer Rating	امتیاز داده شده به مشتری (۰ تا ۵)	DECIMAL(2, 1)
Payment Method	روش پرداخت (Cash, UPI, Credit Card, etc.)	VARCHAR(20)

حالا برای اینکه روابط بین موجودیت ها رو بهتر نشون بدم، یک دیاگرام ER ساده کشیدم که موجودیت‌ها و روابط رو نشون می‌ده. این بر اساس فهم خودم از دیتاست هست:



شکل ۱ دیاگرام موجودیت‌ها و روابط (ER Diagram)

گزارش فاز دوم: Medalion Architecture لایه Bronze

این لایه Raw Layer هست یا داده خام و هدفش ذخیره داده‌ها بدون هیچ تغییری روی اون هاست. داده‌ها رو مستقیم از CSV ایمپورت می‌کنیم و وارد دیتابیس انتخابی که من از PostgreSQL استفاده کردم که یکی از بهترین و محبوب‌ترین دیتابیس‌های Relational هست.

من تمامی مراحل رو کامل در این گزارش می‌نویسم:

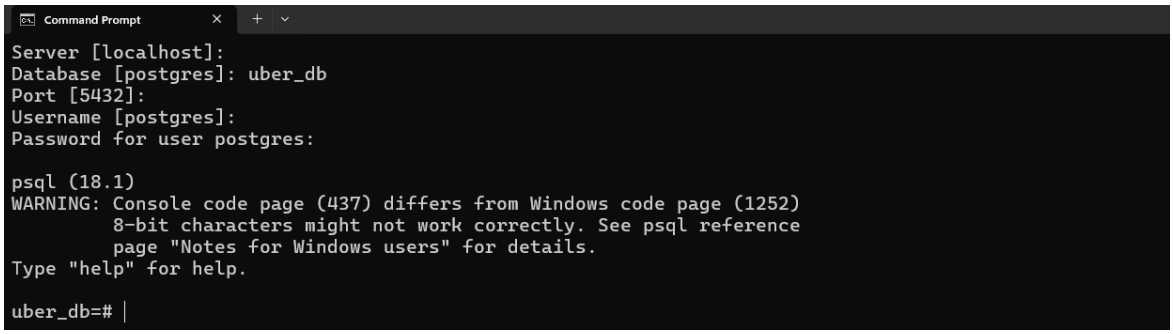
۱. دانلود PostgreSQL از این آدرس که وب‌سایت رسمی پیشنهاد داده:

https://www.enterprisedb.com/downloads/postgres-postgresql-downloads/

این نسخه لوکال روی ماشین ما نصب می‌شود. می‌توانستیم با استفاده از داکر یا روش‌های دیگر هم نصب کنیم ولی من ترجیح دادم روی سیستم و به صورت لوکال نصب کنم.

۲. ساخت جدول در پایگاه داده:

پس از نصب، از Shell psql استفاده می‌کنیم و راه‌اندازی اولیه رو انجام می‌دهیم:



شکل ۲ راه‌اندازی اولیه PostgreSQL از طریق Shell psql

حال با دستور زیر جدول اول‌مان رو ایجاد می‌کنیم:

```
CREATE DATABASE uber_db;
```

۳. حال کافی‌است اسکیمای اولیه برنز و جدول رو بر اساس نوع داده‌هایی کـه در قسـمت اول و فـاز اول تعریف و تحلیل کردیم، تعریف کنیم:

```
CREATE TABLE bronze.raw_dataset (
                                    date DATE,
                                    time TIME,
                                    booking_id VARCHAR(20),
                                    booking_status VARCHAR(50),
                                    customer_id VARCHAR(20),
                                    vehicle_type VARCHAR(50),
                                    cancelled_by_customer INTEGER,
                                    customer_cancel_reason TEXT,
                                    cancelled_by_driver INTEGER,
                                    driver_cancel_reason TEXT,
                                    incomplete_rides INTEGER,
                                    incomplete_reason TEXT,
                                    booking_value DECIMAL(10,2),
                                    ride_distance DECIMAL(10,2),
                                    driver_rating DECIMAL(2,1),
                                    customer_rating DECIMAL(2,1),
                                    payment_method VARCHAR(50)
);
```

این کد صرفا جدول داده‌های Bronze از معماری مدالیون رو با توجه به شناخت و تحلیلی کـه در فـاز و قسمت اول انجام دادیم ایجاد می‌کند. که این کد در محیط psql باید در PostgreSQL اجرا شود.

```
uber_db=# CREATE SCHEMA IF NOT EXISTS bronze;
CREATE SCHEMA
uber_db=#
uber_db=# CREATE TABLE IF NOT EXISTS bronze.raw_dataset (
uber_db(#     date DATE,
uber_db(#     time TIME,
uber_db(#     booking_id VARCHAR(20),
uber_db(#     booking_status VARCHAR(50),
uber_db(#     customer_id VARCHAR(20),
uber_db(#     vehicle_type VARCHAR(50),
uber_db(#     cancelled_by_customer INTEGER,
uber_db(#     customer_cancel_reason TEXT,
uber_db(#     cancelled_by_driver INTEGER,
uber_db(#     driver_cancel_reason TEXT,
uber_db(#     incomplete_rides INTEGER,
uber_db(#     incomplete_reason TEXT,
uber_db(#     booking_value DECIMAL(10,2),
uber_db(#     ride_distance DECIMAL(10,2),
uber_db(#     driver_rating DECIMAL(2,1),
uber_db(#     customer_rating DECIMAL(2,1),
uber_db(#     payment_method VARCHAR(50)
uber_db(# );
CREATE TABLE
uber_db=#
```

شکل ۳ ساختن اسکیمای Bronze و ساخت جدول لایه اول Bronze Layer

۴. کافی است داده‌های خام رو از فایل اولیه که من به original_dataset تغییر نام دادم و داخل پوشه database در پروژه ام قرار دادم ایمپورت کنیم و سپس بدون هیچ تغییری آن را داخل دیتابیس و جدولی که ایجاد کردیم، ذخیره کنیم. برای ایمپورت کردن داده‌ها از فایل CSV، ما یک اسکریپت Node.js می‌نویسیم که در انجام فرایند ETL[1] به ما کمک می‌کند و بخشی از کار اولیه ماست.

نکته حائز اهمیت که خیلی خیلی هم مهم هست اینه که برای درج اطلاعات از فایل CSV به خود دیتابیس و جداول PostgreSQL ، مقادیری مثل "null" رشته‌ای به عنوان string داخل CSV وجود داره که این‌ها رو PostgreSQL نمی‌تونه به عدد یا نوع NULL واقعی تبدیل کنه و این ارور رو قاعدتا خواهیم گرفت:

```
error: invalid input syntax for type integer: "null"
    at D:\Daneshgoh\Master\Term 1\Advance Data Base\T
amrin\Project\uber_data_platform\backend\node_modules
\pg\lib\client.js:624:17
    at process.processTicksAndRejections (node:intern
al/process/task_queues:103:5)
```

شکل ۴ ارور وجود null رشته‌ای در فایل csv.

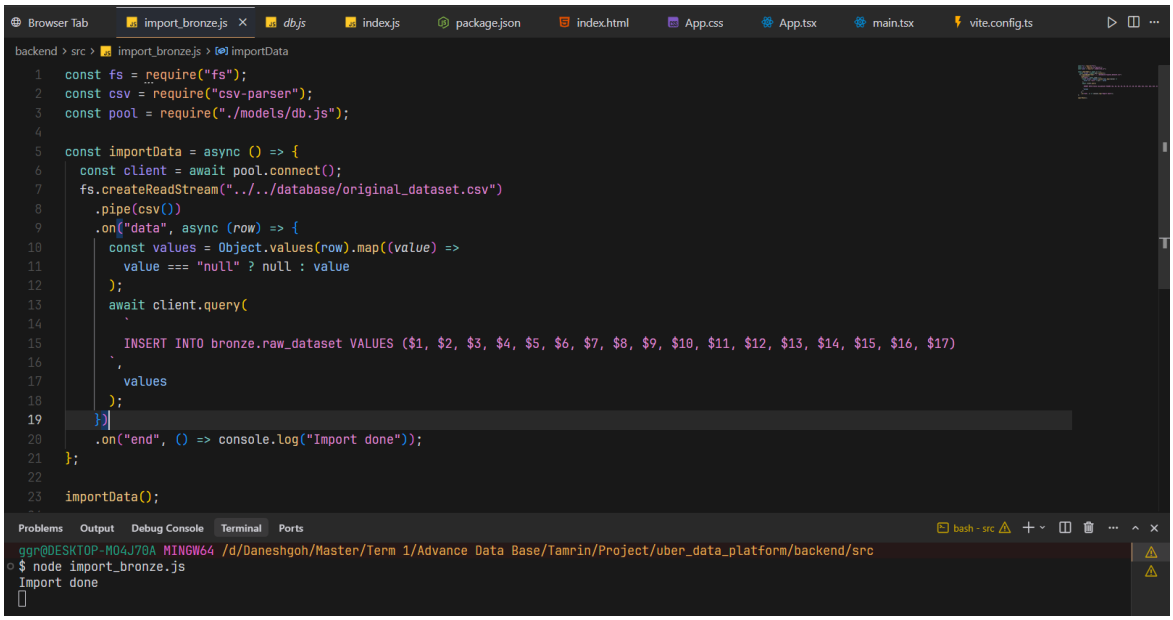
برای همین باید توجه کنیم که کد نیاز به اصلاح داره تا این‌string ها رو به null تبدیل کنه قبل از INSERT کردن.

برای همین این مورد رو به صورت دستی هندل می‌کنیم:

```
const values = Object.values(row).map((value) =>
        value === "null" ? null : value
      );
```

در نهایت کد نهایی که داده‌ها رو از فایل اکسل csv. می‌گیره و داخل Database ایمپورت می‌کنه به صورت زیر نوشته‌شده که در پایان هم پیغام Import don با موفقیت چاپ می‌شه و این به معنای اتمام لایه برنز Bronze است. چون در لایه Bronze پروژه (طبق Medallion Architecture)، هدف Raw Ingestion هست.

[1] Extract Transform Load: به معنی استخراج - انتقال داده - بارگزاری داده است



شکل ۵ کد وارد کردن داده‌های خام از فایل csv به داخل bronze schema و جدول raw_dataset

همچنین اتصال به دیتابیس هم توسط این کد خیلی ساده انجام می‌شود و اتصال به پایگاه‌داده برقراره:

```
const { Pool } = require('pg');
const pool = new Pool({
  user: 'postgres',
  host: 'localhost',
  database: 'uber_db',
  password: 'Admin@123456',
  port: 5432,
});

module.exports = pool;
```

این کد خیلی ساده بک‌اند رو به دیتابیس متصل می‌کند.

```
uber_db=# \conninfo
      Connection Information
      Parameter        |   Value
-----------------------+-----------
 Database              | uber_db
 Client User           | postgres
 Host                  | localhost
 Host Address          | ::1
 Server Port           | 5432
 Options               |
 Protocol Version      | 3.0
 Password Used         | true
 GSSAPI Authenticated  | false
 Backend PID           | 11228
 SSL Connection        | false
 Superuser             | on
 Hot Standby           | off
(13 rows)
```

شکل ۶ چک کردن اتصال به PostgreSQL

خب در ادامه تمامی جداول و اسکیماهایی که داخل postgreSQL ساخته شده‌اند رو نمایش می‌دهیم:

```
uber_db=# \dt *.*
                    List of tables
      Schema       |          Name          |    Type     |  Owner
-------------------+------------------------+-------------+----------
 bronze            | raw_dataset            | table       | postgres
 information_schema | sql_features          | table       | postgres
 information_schema | sql_implementation_info | table      | postgres
 information_schema | sql_parts             | table       | postgres
 information_schema | sql_sizing            | table       | postgres
 pg_catalog        | pg_aggregate           | table       | postgres
```

شکل ۷ اسکیماها و جداول ساخته شده در PostgreSQL DB

در نهایت این هم جدول ما داخل PostgreSQL:

```
uber_db=# \d bronze.raw_dataset
                  Table "bronze.raw_dataset"
        Column         |          Type          | Collation | Nullable | Default
-----------------------+------------------------+-----------+----------+---------
 date                  | date                   |           |          |
 time                  | time without time zone |           |          |
 booking_id            | character varying(20)  |           |          |
 booking_status        | character varying(50)  |           |          |
 customer_id           | character varying(20)  |           |          |
 vehicle_type          | character varying(50)  |           |          |
 cancelled_by_customer | integer                |           |          |
 customer_cancel_reason | text                  |           |          |
 cancelled_by_driver   | integer                |           |          |
 driver_cancel_reason  | text                   |           |          |
 incomplete_rides      | integer                |           |          |
 incomplete_reason     | text                   |           |          |
 booking_value         | numeric(10,2)          |           |          |
 ride_distance         | numeric(10,2)          |           |          |
 driver_rating         | numeric(2,1)           |           |          |
 customer_rating       | numeric(2,1)           |           |          |
 payment_method        | character varying(50)  |           |          |
```

شکل ۸ لایه Bronze و raw_dataset در PostgreSQL DB

گزارش فاز سوم: Medalion Architecture لایه Silver

خب برای انجام این قسمت اول از همه، schema و جدول silver_cleaned_dataset رو می‌سازیم. این جدول همون داده‌های تمیز شده و غنی‌شده‌ست که برای تحلیل و لایهGold قسمت بعد آماده می‌شوند.

با توجه به دیتاست و اهداف مرحله نقره‌ای(Silver) ، تمرکز اصلی بر **تمیزسازی، یکپارچه‌سازی و مهندسی ویژگی** است تا داده‌ها برای تحلیل آماده شوند. در این مرحله نباید رکوردها حذف شوند مگر در موارد بسیار ضروری، زیرا مقادیر NULL در این دیتاست اغلب معنای تجاری دارند و نبود یک رویداد را نشان می‌دهند. من تمام عملیات را در یک تراکنش نوشته‌ام تا اگر خطایی رخ دهد، همه چیز به حالت قبل برگردد. در ادامه هر یک از چهار اقدامی که در صورت پروژه برای این مرحله مشخص شده را به‌همراه کد مربوطه و توضیحات لازم می‌آورم و به آن پرداخته خواهد شد.

۹-۲- راه‌اندازی اولیه تراکنش

```
require('dotenv').config();
const pool = require('../models/db.js');
async function runSilverETL() {
  const client = await pool.connect();
  try {
    console.log('🚀 Starting Silver Layer ETL ...');
    await client.query('BEGIN');
    // ... Implementation
    await client.query('COMMIT');
    console.log('🎉 Silver Layer ETL completed successfully!');
  } catch (err) {
    await client.query('ROLLBACK');
    console.error('❌ Silver Layer ETL failed:', err);
    throw err;
  } finally {
    client.release();
  }
}
runSilverETL().catch(console.error);
```

خب اول از همه کتابخانه‌های لازم رو استفاده کردیم dotenv رو برای خوندن متغیرهای محیطی (مثل اطلاعات دیتابیس) استفاده می‌کنیم. بعدش با pool از فایل db.js یه کانکشن از دیتابیس می‌گیرم. کل عملیات رو توی یه try/catch گذاشتیم که اگه خطایی پیش بیاد، تراکنش رو ROLLBACK و اگه همه چیز طبق خواسته ما پیش‌بره، در انتها COMMIT می‌کنیم. این کار باعث میشه داده‌ها همیشه در یک وضعیت consistent و سازگار باشند. لاگ‌ها رو هم با پیغام مناسب ایجاد کردیم.

۱-۲- اقدام اول مهندسی ویژگی (Feature Engineering)

اصلا مهندسی ویژگی یعنی چه؟ فرآیند تبدیل داده‌های خام به متغیرها (ویژگی‌هایی) است که بهتر بتوانند ماهیت بیزینس را به نمایش بگذارند. در واقع ما داده‌ها را دستکاری می‌کنیم تا هوشمندتر شوند و در مراحل بعدی (مثل تحلیل در داشبورد یا پرسش‌وپاسخ هوشمند) نتایج دقیق‌تری بگیریم.

```
-- Create silver schema and cleaned_dataset table with feature-engineered columns
CREATE SCHEMA IF NOT EXISTS silver;
CREATE TABLE IF NOT EXISTS silver.cleaned_dataset (
    booking_id           TEXT PRIMARY KEY,
    customer_id          TEXT,
    vehicle_type         TEXT,
    booking_status       TEXT,
    booking_value        NUMERIC(10,2),
    ride_distance        NUMERIC(10,2),
    payment_method       TEXT,
    trip_timestamp       TIMESTAMP,
    pickup_hour          INTEGER,
    day_of_week          INTEGER,   -- 0=Sunday, 6=Saturday
    day_name             TEXT,      -- Monday, Tuesday, ...
    month                INTEGER,
    year                 INTEGER,
    is_weekend           BOOLEAN,
    unified_cancellation_reason TEXT,
    driver_rating        NUMERIC(3,2),
    customer_rating      NUMERIC(3,2),
    driver_rating_imputed  BOOLEAN DEFAULT FALSE,
    customer_rating_imputed BOOLEAN DEFAULT FALSE,
    is_cancelled         BOOLEAN
);
```

ابتدا اسکیمای silver رو ساختیم. سپس جدول اصلی یعنی cleaned_dataset رو ایجاد کرده‌ایم. ستون‌هایی که از Bronze می‌آیند رو با DataType مناسب تعریف کرده‌ایم. چند ستون جدید هم برای مهندسی ویژگی اضافه کردیم که در زیر توضیح می‌دهیم:

اولین چیزی که توی دیتاست برنز به چشم می‌خورد، جدا بودن ستون‌های Date و Time با فرمت غیراستاندارد است. برای اینکه بتونیم تحلیل‌های زمانی راحت‌تری انجام بدهیم. باید این دو تا ستون رو باهم ترکیب کنیم و به یه تایم‌استمپ واقعی تبدیل کنیم.

trip_timestam که ترکیب تاریخ و زمان هست، طبق اشاره صورت پروژه و از اون pickup_hour برای ساعت سفر و روز هفته day_of_week به صورت عددی برای مثال 0 = Saturday، نام روز day_name (مانند:Monday) و month و year استخراج می‌شوند. is_weekend هم که نشون میده سفر آخر هفته بوده یا نه.

اینا همه کمک می‌کنند تا در مراحل بعدی دیگه نیازی به دستکاری تاریخ و زمان نداشته باشیم. دو ستون driver_rating_imputed و customer_rating_imputed نیز با مقدار پیش‌فرض FALSE گذاشتیم تا بعداً وقتی امتیازها رو جایگزین کردیم، پرچم‌داشته باشند و مشخص باشه کدوم مقادیر رو ما خودمون جایگزین کرده‌بودیم. ستون is_cancelled رو هم به صورت بولین اضافه کردم تا خیلی راحت بتونم سفرهای لغو شده رو فیلتر کنم.

۱-۳-۲- پاکسازی داده‌های قبلی و درج اولیه از برنز

```
TRUNCATE silver.cleaned_dataset;
```

قبل از اینکه داده‌های جدید رو وارد کنم، با TRUNCATEجدول نقره‌ای رو خالی کردم تا مطمئن بشم داده‌های تکراری و قدیمی نداشته باشیم.

```
INSERT INTO silver.cleaned_dataset (
    booking_id, customer_id, vehicle_type, booking_status,
    booking_value, ride_distance, payment_method,
    trip_timestamp,
    pickup_hour, day_of_week, day_name, month, year, is_weekend,
    unified_cancellation_reason,
    driver_rating, customer_rating,
    is_cancelled
)
SELECT
    "Booking ID",
    "Customer ID",
    "Vehicle Type",
    "Booking Status",
    NULLIF("Booking Value", '')::NUMERIC,
    NULLIF("Ride Distance", '')::NUMERIC,
    "Payment Method",
    TO_TIMESTAMP("Date" || ' ' || "Time", 'MM/DD/YYYY HH24:MI:SS'),
    EXTRACT(HOUR FROM TO_TIMESTAMP("Date" || ' ' || "Time", 'MM/DD/YYYY HH24:MI:SS')),
    EXTRACT(DOW FROM TO_TIMESTAMP("Date" || ' ' || "Time", 'MM/DD/YYYY HH24:MI:SS')),
    TO_CHAR(TO_TIMESTAMP("Date" || ' ' || "Time", 'MM/DD/YYYY HH24:MI:SS'), 'FMDay'),
    EXTRACT(MONTH FROM TO_TIMESTAMP("Date" || ' ' || "Time", 'MM/DD/YYYY HH24:MI:SS')),
    EXTRACT(YEAR FROM TO_TIMESTAMP("Date" || ' ' || "Time", 'MM/DD/YYYY HH24:MI:SS')),
    CASE WHEN EXTRACT(DOW FROM TO_TIMESTAMP("Date" || ' ' || "Time", 'MM/DD/YYYY HH24:MI:SS')) IN (0, 6)
         THEN TRUE ELSE FALSE END,
```

برای درج دستور INSERT رو نوشتم. اینجا از NULLIF استفاده کردم تا رشته‌های خالی که ممکنه توی ستون‌های عددی باشن به NULL تبدیل بشن و موقع تبدیل به عدد خطا ندهند سپس با TO_TIMESTAMP و مشخص کردن دقیق فرمت ورودی، تاریخ و زمان رو به یه تایم‌استمپ استاندارد تبدیل کردم. همون‌جا و بلا فاصله ویژگی‌های pickup_hour, day_of_week, day_name, month, year و is_weekend رو استخراج کردم.

۲-۲- اقدام دوم: یکپارچه‌سازی دلایل لغو سفر

سه ستون جداگانه برای دلیل لغو داشتیم که بسته به وضعیت سفر پر می‌شدند. من یک ستون unified_cancellation_reason با توجه به صورت پروژه ساختم که همه دلایل رو در یک جا جمع می‌کنه بدون نیاز به خواندن چندین ستون. من به جای اینکه به booking_status تکیه کنم و با ILIKE رشته‌ها رو مقایسه کنم (که ممکنه توی داده‌های واقعی جواب نده)، تصمیم گرفتم مستقیم برم سراغ خود ستون‌های نشانگر لغو. یعنی اگه ستون Cancelled Rides by Customer غیرتهی بود، یعنی دلیل لغو مشتری توی ستون بعدش وجود داره. همین منطق رو برای راننده و سفر ناتمام هم به کار بردم. به این ترتیب یه ستون واحد unified_cancellation_reason ساخته‌شد:

```
-- -- Unified cancellation reason: pick the reason from the column that has a non-null indicator
    CASE
        WHEN "Cancelled Rides by Customer" IS NOT NULL THEN "Reason for cancelling by Customer"
        WHEN "Cancelled Rides by Driver"   IS NOT NULL THEN "Driver Cancellation Reason"
        WHEN "Incomplete Rides"            IS NOT NULL THEN "Incomplete Rides Reason"
        ELSE "Reason Not Specified"
    END,
```

نکته حائز اهمیت این است که برای جلوگیری از افزایش تعداد null ها در ستون جدید ما اگه دلیل کنسل شدن سفر رو نداشته باشیم: null نمی‌گذاریم! بلکه با عبارت "Reason Not Specified" اون رو پر می‌کنیم. این کار معمولا در Data Warehouse خیلی بهتر است. این کار هم جدول رو جمع‌وجورتر کرده و هم کوئری‌زدن برای تحلیل دلایل لغو رو فوق‌العاده ساده‌تر می‌کنه.

این کار همچنین یکی از ایده‌های مدیریت مقادیر null هم هست که در قسمت بعدی خواسته شده بود.

۲-۳- اقدام سوم: مدیریت مقادیر Null

وقتی داده‌های برنز رو بررسی کردم، دیدم که ستون‌هایی مثل cancelled_by_customer، cancelled_by_driver، incomplete_rides و حتی booking_value و ride_distance کلی مقدار NULL دارند. مهم‌تر از همه، بعضی از رشته‌های خالی هم وجود داشت که

موقع تبدیل به عدد باعث خطا می‌شد. اولین کاری که کردم این بود که با NULLIF مطمئن شدم رشته‌های خالی واقعاً به NULL تبدیل بشن، نه اینکه خطا بخواهیم بدیم!

اما نکته اصلی اینه که من **حتی یک ردیف رو هم حذف نکردم**. چرا؟ چون توی این دیتاست، NULL بودن خیلی از ستون‌ها معنای تجاری داره. مثلاً وقتی سفر کنسل شده، طبیعی‌ست که مسافت یا مبلغ سفر ثبت نشده باشه.

```
-- Convert empty strings to NULL for numeric fields
NULLIF("Booking Value", '')::NUMERIC AS booking_value,
NULLIF("Ride Distance", '')::NUMERIC AS ride_distance,
```

و تنها جایی که واقعاً باید فکری به حال NULL ها می‌کردم، ستون‌های امتیاز بود که اونم توی اقدام چهارم جداگانه حلش کردم. خلاصه اینکه استراتژی من در مدیریت NULL این بود که:

NULL های منطقی رو حفظ کن، فقط اونایی رو که میشه با یه مقدار معقول پر کرد (مثل امتیاز) رو جایگزین کن، و هرگز ردیف رو پاک نکن.

۴-۲- اقدام چهارم: تحلیل و مدیریت امتیازها

بزگترین چالش این مرحله، ستون‌های driver_rating و customer_rating بودند.

در سفرهای کامل‌شده، تعداد زیادی NULL دیدم که نشون می‌داد راننده یا مشتری بعد از اصلا نخواسته امتیاز بده و نه اینکه داده‌ها مشکل داشته باشند همچنین: در صورت پروژه صراحتاً گفته بود «از حذف ردیف‌ها به دلیل نبودن امتیاز خودداری کنید». پس راهی که به نظر می‌رسه پر کردن سلول ها با مقادیر جدید هست.

اول فکر کردم یک میانگین کلی بگیرم، ولی بعد گفتم این کار عادلانه نیست؛ چون ممکنه خودروهای لوکس امتیاز بالاتری داشته باشن و خودروهای اقتصادی پایین‌تر و اگه که میانگین کلی رو جایگزین کنم، تحلیل‌های بعدی رو به هم می‌ریزم. برای همین رفتم سراغ محاسبه میانگین **به تفکیک نوع خودرو (vehicle_type)** . در کوئری زیر با WITH میانگین هر خودرو رو حساب کردم و بعد توی UPDATE، اگه امتیاز NULL بود، مقدار میانگین همون نوع خودرو رو گذاشتم. حالا اگر هم

برحسب اتفاق میانگین هم NULL می‌شد (یعنی هیچ امتیازی برای اون خودرو ثبت نشده بود)، از یه مقدار پیش‌فرض ۴.۵ استفاده کردم.که در داده های uber معمولا میانگین همین تقریبا نزدیکه به همین عدد است این منبع تایید میکنه:https://www.uber.com/en-AU/newsroom/riderratingsranking2024/

دو تا ستون بولین هم به اسم driver_rating_imputed و customer_rating_imputed اضافه کردم تا مشخص بشه کدوم امتیازها واقعی‌اند و کدوم‌ها جایگزین شدن. این کار هم مشکل NULL رو حل کرد، هم دقت رو بالاتر برد. کدهای این بخش رو هم آوردم:

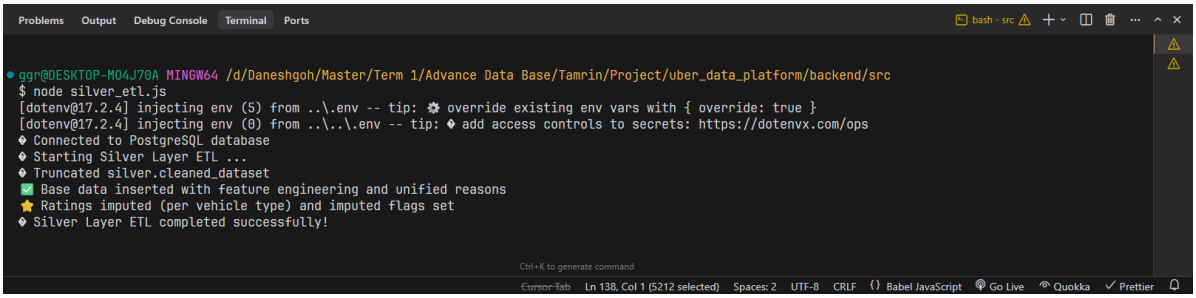
```
-- Calculate average ratings per vehicle type for completed trips only
WITH avg_per_vehicle AS (
    SELECT
        vehicle_type,
        AVG(driver_rating) FILTER (
            WHERE booking_status = 'Completed' AND driver_rating IS NOT NULL
        ) AS avg_driver,
        AVG(customer_rating) FILTER (
            WHERE booking_status = 'Completed' AND customer_rating IS NOT NULL
        ) AS avg_customer
    FROM silver.cleaned_dataset
    GROUP BY vehicle_type
)
-- Update null ratings with vehicle-specific averages and set imputed flags
UPDATE silver.cleaned_dataset s
SET
    driver_rating = COALESCE(s.driver_rating, a.avg_driver, 4.5),
    customer_rating = COALESCE(s.customer_rating, a.avg_customer, 4.5),
    driver_rating_imputed = CASE
        WHEN s.driver_rating IS NULL AND s.booking_status = 'Completed' THEN TRUE
        ELSE FALSE
    END,
    customer_rating_imputed = CASE
        WHEN s.customer_rating IS NULL AND s.booking_status = 'Completed' THEN TRUE
        ELSE FALSE
    END
FROM avg_per_vehicle a
WHERE s.vehicle_type = a.vehicle_type
```

```
AND s.booking_status = 'Completed';
```

۵-۲- پایان تراکنش و لاگ نهایی

اگر که همه‌ی این مراحل با موفقیت انجام بشن، COMMIT می‌شه تراکنش و پیام موفقیت چاپ می‌شه. در غیر این صورت ROLLBACK و خطا نشون داده می‌شه و سیستم به وضعیت قبلی خود به طور کامل برمی‌گرده.

کد کامل که تمامی اقدامات رو دربرگرفته رو اجرا می‌کنیم و مرحله به مرحله گزارش می‌دهیم:



شکل ۹ اجرای موفق کد silver_etl و لایه silver

برای مقایسه بهتر و مشاهده تفاوت‌ها ابتدا ساختار جدول Bronze رو نشان می‌دهیم:

```
uber_db=# \d bronze.raw_dataset
                    Table "bronze.raw_dataset"
        Column         |          Type          | Collation | Nullable | Default
-----------------------+------------------------+-----------+----------+---------
 date                  | date                   |           |          |
 time                  | time without time zone |           |          |
 booking_id            | character varying(20)  |           |          |
 booking_status        | character varying(50)  |           |          |
 customer_id           | character varying(20)  |           |          |
 vehicle_type          | character varying(50)  |           |          |
 cancelled_by_customer | integer                |           |          |
 customer_cancel_reason| text                   |           |          |
 cancelled_by_driver   | integer                |           |          |
 driver_cancel_reason  | text                   |           |          |
 incomplete_rides      | integer                |           |          |
 incomplete_reason     | text                   |           |          |
 booking_value         | numeric(10,2)          |           |          |
 ride_distance         | numeric(10,2)          |           |          |
 driver_rating         | numeric(2,1)           |           |          |
 customer_rating       | numeric(2,1)           |           |          |
 payment_method        | character varying(50)  |           |          |
```

شکل ۱۰ ساختار جدول Bronze

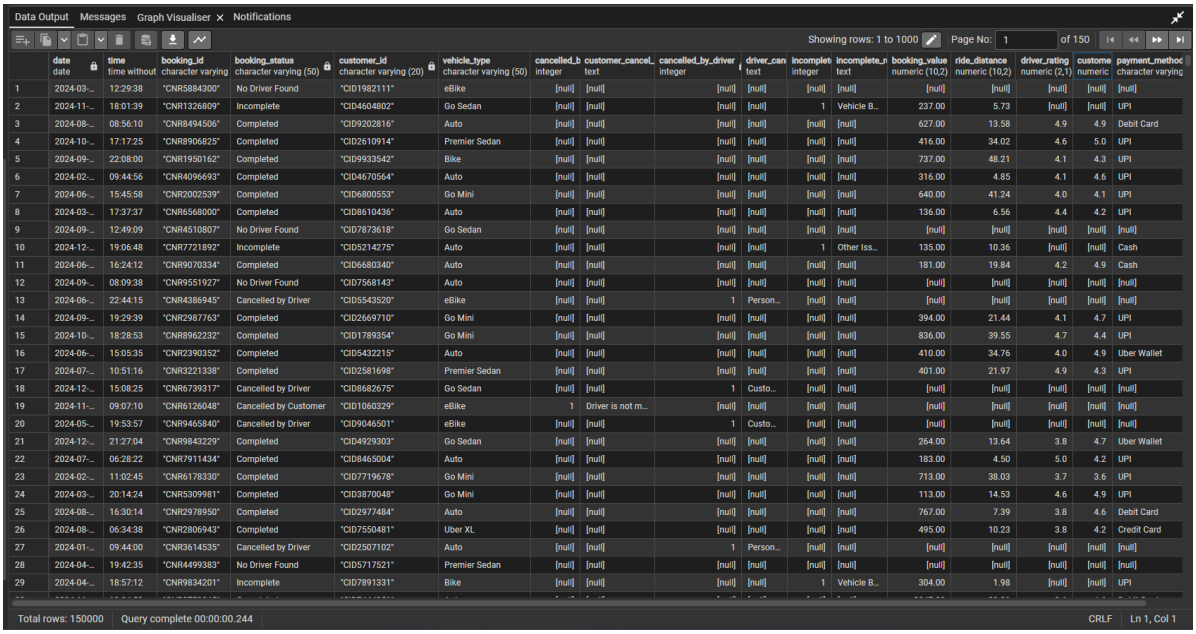
```
uber_db-# \d silver.cleaned_dataset
                        Table "silver.cleaned_dataset"
          Column          |            Type             | Collation | Nullable | Default
--------------------------+-----------------------------+-----------+----------+---------
 booking_id               | text                        |           | not null |
 customer_id              | text                        |           |          |
 vehicle_type             | text                        |           |          |
 booking_status           | text                        |           |          |
 booking_value            | numeric(10,2)               |           |          |
 ride_distance            | numeric(10,2)               |           |          |
 payment_method           | text                        |           |          |
 trip_timestamp           | timestamp without time zone |           |          |
 pickup_hour              | integer                     |           |          |
 day_of_week              | integer                     |           |          |
 day_name                 | text                        |           |          |
 month                    | integer                     |           |          |
 year                     | integer                     |           |          |
 is_weekend               | boolean                     |           |          |
 unified_cancellation_reason | text                     |           |          |
 driver_rating            | numeric(3,2)                |           |          |
 customer_rating          | numeric(3,2)                |           |          |
 driver_rating_imputed    | boolean                     |           |          | false
 customer_rating_imputed  | boolean                     |           |          | false
 is_cancelled             | boolean                     |           |          |
Indexes:
    "cleaned_dataset_pkey" PRIMARY KEY, btree (booking_id)
```

New Columns

شکل ۱۱ ساختار جدول Silver

در ادامه نمونه‌داده‌های دو جدول رو مقایسه می‌کنیم:





Data Output | Messages | Graph Visualiser | Notifications

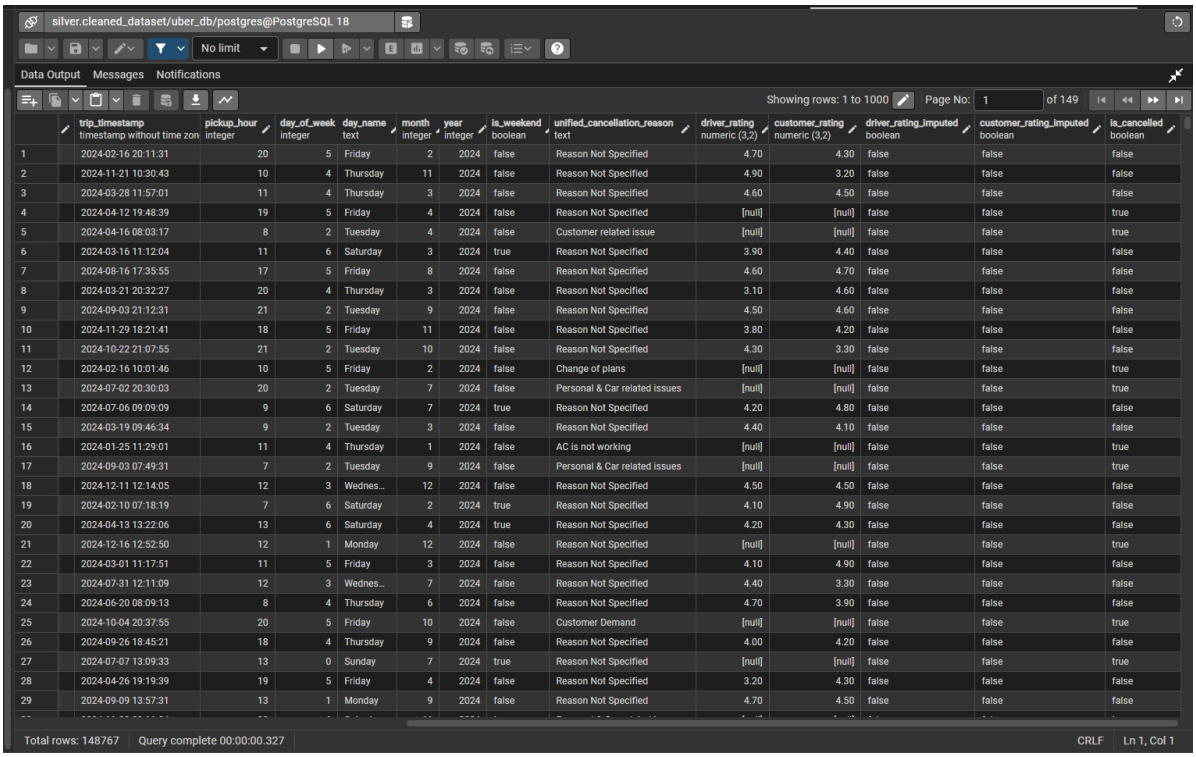
Showing rows: 1 to 1000 | Page No: 1 of 150

	date	time	booking_id	booking_status	customer_id	vehicle_type	cancelled_b	customer_cancel_	cancelled_by_driver	driver_can	incomplet	incomplete_r	booking_value	ride_distance	driver_rating	custome	payment_metho
	date	time without	character varying	character varying (50)	character varying (20)	character varying (50)	integer	text	integer	text	integer	text	numeric (10,2)	numeric (10,2)	numeric (2,1)	numeric	character varying
1	2024-03-...	12:29:38	"CNR5884300"	No Driver Found	"CID1982111"	eBike	[null]	[null]	[null]	[null]	[null]	[null]	[null]	[null]	[null]	[null]	[null]
2	2024-11-...	18:01:39	"CNR1326809"	Incomplete	"CID4604802"	Go Sedan	[null]	[null]	[null]	[null]	1	Vehicle B...	237.00	5.73	[null]	[null]	UPI
3	2024-08-...	08:56:10	"CNR8494506"	Completed	"CID9202816"	Auto	[null]	[null]	[null]	[null]	[null]	[null]	627.00	13.58	4.9	4.9	Debit Card
4	2024-10-...	17:17:25	"CNR8906825"	Completed	"CID2610914"	Premier Sedan	[null]	[null]	[null]	[null]	[null]	[null]	416.00	34.02	4.6	5.0	UPI
5	2024-09-...	22:08:00	"CNR1950162"	Completed	"CID9933542"	Bike	[null]	[null]	[null]	[null]	[null]	[null]	737.00	48.21	4.1	4.3	UPI
6	2024-02-...	09:44:56	"CNR4096693"	Completed	"CID4670564"	Auto	[null]	[null]	[null]	[null]	[null]	[null]	316.00	4.85	4.1	4.6	UPI
7	2024-06-...	15:45:58	"CNR2002539"	Completed	"CID6800553"	Go Mini	[null]	[null]	[null]	[null]	[null]	[null]	640.00	41.24	4.0	4.1	UPI
8	2024-03-...	17:37:37	"CNR6568000"	Completed	"CID8610436"	Auto	[null]	[null]	[null]	[null]	[null]	[null]	136.00	6.56	4.4	4.2	UPI
9	2024-09-...	12:49:09	"CNR4510807"	No Driver Found	"CID7873618"	Go Sedan	[null]	[null]	[null]	[null]	[null]	[null]	[null]	[null]	[null]	[null]	[null]
10	2024-12-...	19:06:48	"CNR7721892"	Incomplete	"CID5214275"	Auto	[null]	[null]	[null]	[null]	1	Other Iss...	135.00	10.36	[null]	[null]	Cash
11	2024-06-...	16:24:12	"CNR9070334"	Completed	"CID6680340"	Auto	[null]	[null]	[null]	[null]	[null]	[null]	181.00	19.84	4.2	4.9	Cash
12	2024-09-...	08:09:38	"CNR9551927"	No Driver Found	"CID7568143"	Auto	[null]	[null]	[null]	[null]	[null]	[null]	[null]	[null]	[null]	[null]	[null]
13	2024-06-...	22:44:15	"CNR4386945"	Cancelled by Driver	"CID5543520"	eBike	[null]	[null]	1	Person...	[null]	[null]	[null]	[null]	[null]	[null]	[null]
14	2024-09-...	19:29:39	"CNR2987763"	Completed	"CID2669710"	Go Mini	[null]	[null]	[null]	[null]	[null]	[null]	394.00	21.44	4.1	4.7	UPI
15	2024-10-...	18:28:53	"CNR8962232"	Completed	"CID1789354"	Go Mini	[null]	[null]	[null]	[null]	[null]	[null]	836.00	39.55	4.7	4.4	UPI
16	2024-06-...	15:05:35	"CNR2390352"	Completed	"CID5432215"	Auto	[null]	[null]	[null]	[null]	[null]	[null]	410.00	34.76	4.0	4.9	Uber Wallet
17	2024-07-...	10:51:16	"CNR3221338"	Completed	"CID2581698"	Premier Sedan	[null]	[null]	[null]	[null]	[null]	[null]	401.00	21.97	4.9	4.3	UPI
18	2024-12-...	15:08:25	"CNR6739317"	Cancelled by Driver	"CID8682675"	Go Sedan	[null]	[null]	1	Custo...	[null]	[null]	[null]	[null]	[null]	[null]	[null]
19	2024-11-...	09:07:10	"CNR6126048"	Cancelled by Customer	"CID1060329"	eBike	1	Driver is not m...	[null]	[null]	[null]	[null]	[null]	[null]	[null]	[null]	[null]
20	2024-05-...	19:53:57	"CNR9465840"	Cancelled by Driver	"CID9046501"	eBike	[null]	[null]	1	Custo...	[null]	[null]	[null]	[null]	[null]	[null]	[null]
21	2024-12-...	21:27:04	"CNR9843229"	Completed	"CID4929303"	Go Sedan	[null]	[null]	[null]	[null]	[null]	[null]	264.00	13.64	3.8	4.7	Uber Wallet
22	2024-07-...	06:28:22	"CNR7911434"	Completed	"CID8465004"	Auto	[null]	[null]	[null]	[null]	[null]	[null]	183.00	4.50	5.0	4.2	UPI
23	2024-02-...	11:02:45	"CNR6178330"	Completed	"CID7719678"	Go Mini	[null]	[null]	[null]	[null]	[null]	[null]	713.00	38.03	3.7	3.6	UPI
24	2024-03-...	20:14:24	"CNR5309981"	Completed	"CID3870048"	Go Mini	[null]	[null]	[null]	[null]	[null]	[null]	113.00	14.53	4.6	4.9	UPI
25	2024-08-...	16:30:14	"CNR2978950"	Completed	"CID2977484"	Auto	[null]	[null]	[null]	[null]	[null]	[null]	767.00	7.39	3.8	4.6	Debit Card
26	2024-08-...	06:34:38	"CNR2806943"	Completed	"CID7550481"	Uber XL	[null]	[null]	[null]	[null]	[null]	[null]	495.00	10.23	3.8	4.2	Credit Card
27	2024-01-...	09:44:00	"CNR3614535"	Cancelled by Driver	"CID2507102"	Auto	[null]	[null]	1	Person...	[null]	[null]	[null]	[null]	[null]	[null]	[null]
28	2024-04-...	19:42:35	"CNR4499383"	No Driver Found	"CID5717521"	Premier Sedan	[null]	[null]	[null]	[null]	[null]	[null]	[null]	[null]	[null]	[null]	[null]
29	2024-04-...	18:57:12	"CNR9834201"	Incomplete	"CID7891331"	Bike	[null]	[null]	[null]	[null]	1	Vehicle B...	304.00	1.98	[null]	[null]	UPI

Total rows: 150000 | Query complete 00:00:00.244 | CRLF | Ln 1, Col 1

شکل ۱۲ داده‌های لایه برنز قبل از تغییرات در pgAdmin4

در ادامه اسکیمای Silver و جدول silver_cleaned_dataset رو می‌بینیم:



Data Output | Messages | Notifications

Showing rows: 1 to 1000 | Page No: 1 of 149

	trip_timestamp timestamp without time zon	pickup_hour integer	day_of_week integer	day_name text	month integer	year integer	is_weekend boolean	unified_cancellation_reason text	driver_rating numeric (3,2)	customer_rating numeric (3,2)	driver_rating_imputed boolean	customer_rating_imputed boolean	is_cancelled boolean
1	2024-02-16 20:11:31	20	5	Friday	2	2024	false	Reason Not Specified	4.70	4.30	false	false	false
2	2024-11-21 10:30:43	10	4	Thursday	11	2024	false	Reason Not Specified	4.90	3.20	false	false	false
3	2024-03-28 11:57:01	11	4	Thursday	3	2024	false	Reason Not Specified	4.60	4.50	false	false	false
4	2024-04-12 19:48:39	19	5	Friday	4	2024	false	Reason Not Specified	[null]	[null]	false	false	true
5	2024-04-16 08:03:17	8	2	Tuesday	4	2024	false	Customer related issue	[null]	[null]	false	false	true
6	2024-03-16 11:12:04	11	6	Saturday	3	2024	true	Reason Not Specified	3.90	4.40	false	false	false
7	2024-08-16 17:35:55	17	5	Friday	8	2024	false	Reason Not Specified	4.60	4.70	false	false	false
8	2024-03-21 20:32:27	20	4	Thursday	3	2024	false	Reason Not Specified	3.10	4.60	false	false	false
9	2024-09-03 21:12:31	21	2	Tuesday	9	2024	false	Reason Not Specified	4.50	4.60	false	false	false
10	2024-11-29 18:21:41	18	5	Friday	11	2024	false	Reason Not Specified	3.80	4.20	false	false	false
11	2024-10-22 21:07:55	21	2	Tuesday	10	2024	false	Reason Not Specified	4.30	3.30	false	false	false
12	2024-02-16 10:01:46	10	5	Friday	2	2024	false	Change of plans	[null]	[null]	false	false	true
13	2024-07-02 20:30:03	20	2	Tuesday	7	2024	false	Personal & Car related issues	[null]	[null]	false	false	true
14	2024-07-06 09:09:09	9	6	Saturday	7	2024	true	Reason Not Specified	4.20	4.80	false	false	false
15	2024-03-19 09:46:34	9	2	Tuesday	3	2024	false	Reason Not Specified	4.40	4.10	false	false	false
16	2024-01-25 11:29:01	11	4	Thursday	1	2024	false	AC is not working	[null]	[null]	false	false	true
17	2024-09-03 07:49:31	7	2	Tuesday	9	2024	false	Personal & Car related issues	[null]	[null]	false	false	true
18	2024-12-11 12:14:05	12	3	Wednes...	12	2024	false	Reason Not Specified	4.50	4.50	false	false	false
19	2024-02-10 07:18:19	7	6	Saturday	2	2024	true	Reason Not Specified	4.10	4.90	false	false	false
20	2024-04-13 13:22:06	13	6	Saturday	4	2024	true	Reason Not Specified	4.20	4.30	false	false	false
21	2024-12-16 12:52:50	12	1	Monday	12	2024	false	Reason Not Specified	[null]	[null]	false	false	true
22	2024-03-01 11:17:51	11	5	Friday	3	2024	false	Reason Not Specified	4.10	4.90	false	false	false
23	2024-07-31 12:11:09	12	3	Wednes...	7	2024	false	Reason Not Specified	4.40	3.30	false	false	false
24	2024-06-20 08:09:13	8	4	Thursday	6	2024	false	Reason Not Specified	4.70	3.90	false	false	false
25	2024-10-04 20:37:55	20	5	Friday	10	2024	false	Customer Demand	[null]	[null]	false	false	true
26	2024-09-26 18:45:21	18	4	Thursday	9	2024	false	Reason Not Specified	4.00	4.20	false	false	false
27	2024-07-07 13:09:33	13	0	Sunday	7	2024	true	Reason Not Specified	[null]	[null]	false	false	true
28	2024-04-26 19:19:39	19	5	Friday	4	2024	false	Reason Not Specified	3.20	4.30	false	false	false
29	2024-09-09 13:57:31	13	1	Monday	9	2024	false	Reason Not Specified	4.70	4.50	false	false	false

Total rows: 148767 | Query complete 00:00:00.327 | CRLF | Ln 1, Col 1

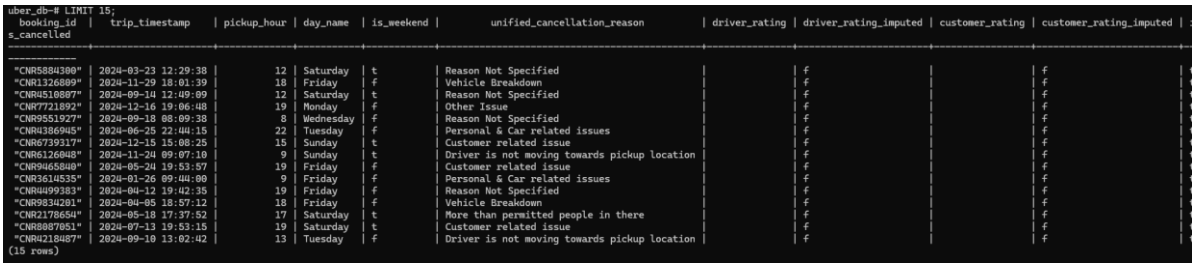
شکل ۱۳ داده‌ها در لایه Silver بعد از تغییرات در pgAdmin4

خیلی واضح است که مقادیر Null بخوبی مدیریت شده و به شدت کاهش‌یافته‌اند. برای نشان‌دادن بهتر این موضوع ما یک کوئری پیچیده نوشتیم که مقادیر Null در دو لایه Bronze و Silver را به همراه درصد بهبود نشان می‌دهد که نتیجه آن را اینجا می‌اوریم:

```
uber_db-#     s.total_null_values AS "Null Count",
uber_db-#     (b.total_null_values - s.total_null_values) AS "Nulls Removed",
uber_db-#     ROUND(((b.total_null_values - s.total_null_values)::NUMERIC / NULLIF(b.total_null_values, 0)) * 100, 2)
rovement"
uber_db-# FROM bronze_stats b, silver_stats s;
       Layer       | Null Count |         Layer          | Null Count | Nulls Removed | Data Quality Improvement
-------------------+------------+------------------------+------------+---------------+--------------------------
 Bronze (Raw Data) |    1065000 | Silver (Cleaned Data) |     313500 |        751500 | 70.56%
(1 row)
```

شکل ۱۴ مقایسه null value ها در دو لایه نقره و برنز و درصد بهبود

با توجه به حجم دیتا (۱۵۰,۰۰۰ ردیف) و تعداد ستون‌ها (حدود ۱۷ ستون در برنز)، کاهش به ۳۱۳ هزار یعنی میانگین NULL به ازای هر ردیف از حدود ۷ به ۲ رسیده. این نشون میده پالایش داده‌ها خیلی مؤثر بوده است. و نسبت بهبود **۷۰٪** گویای این موضوع است.



booking_id	trip_timestamp	pickup_hour	day_name	is_weekend	unified_cancellation_reason	driver_rating	driver_rating_imputed	customer_rating	customer_rating_imputed	s_cancelled
"CNR5884300"	2024-03-23 12:29:38	12	Saturday	t	Reason Not Specified		f		f	
"CNR1326809"	2024-11-29 18:01:39	18	Friday	f	Vehicle Breakdown		f		f	
"CNR4510807"	2024-09-14 12:49:09	12	Saturday	t	Reason Not Specified		f		f	
"CNR7721892"	2024-12-16 19:06:48	19	Monday	f	Other Issue		f		f	
"CNR9851927"	2024-09-18 08:09:38	8	Wednesday	f	Reason Not Specified		f		f	
"CNR4386945"	2024-06-25 22:44:15	22	Tuesday	f	Personal & Car related issues		f		f	
"CNR6739317"	2024-12-15 15:08:25	15	Sunday	t	Customer related issue		f		f	
"CNR6126048"	2024-11-24 09:07:10	9	Sunday	t	Driver is not moving towards pickup location		f		f	
"CNR9465840"	2024-05-24 19:53:57	19	Friday	f	Customer related issue		f		f	
"CNR3614535"	2024-01-26 09:44:00	9	Friday	f	Personal & Car related issues		f		f	
"CNR4499383"	2024-04-12 19:42:35	19	Friday	f	Reason Not Specified		f		f	
"CNR9834201"	2024-04-05 18:57:12	18	Friday	f	Vehicle Breakdown		f		f	
"CNR2178654"	2024-05-18 17:37:52	17	Saturday	t	More than permitted people in there		f		f	
"CNR8087051"	2024-07-13 19:53:15	19	Saturday	t	Customer related issue		f		f	
"CNR4218487"	2024-09-10 13:02:42	13	Tuesday	f	Driver is not moving towards pickup location		f		f	

(15 rows)

شکل ۱۵ جدول Silver و مدیریت مقادیر Null

بررسی عدم وجود Null در ستون جدید unified_cancellation_reason با یک کوئری ساده:

```sql
SELECT
    COUNT(*) AS total_rows,
    COUNT(*) FILTER (WHERE unified_cancellation_reason IS NULL) AS null_count
FROM silver.cleaned_dataset;
```

```
uber_db=# SELECT
uber_db-#     COUNT(*) AS total_rows,
uber_db-#     COUNT(*) FILTER (WHERE unified_cancellation_reason IS NULL) AS null_count
uber_db-# FROM silver.cleaned_dataset;
 total_rows | null_count
------------+------------
     148767 |          0
(1 row)
```

شکل ۱۶ عدم وجود Null در ستون جدید unified_cancellation_reason

در ادامه نیز دلایل لغو رو گزارش می‌کنیم:

```sql
SELECT
    unified_cancellation_reason,
    COUNT(*) AS frequency
FROM silver.cleaned_dataset
GROUP BY unified_cancellation_reason
ORDER BY frequency DESC;
```

unified_cancellation_reason	frequency
Reason Not Specified	102649
Customer related issue	6777
The customer was coughing/sick	6693
Personal & Car related issues	6675
More than permitted people in there	6644
Customer Demand	3015
Vehicle Breakdown	2993
Other Issue	2919
Wrong Address	2348
Change of plans	2326
Driver is not moving towards pickup location	2315
Driver asked to cancel	2274
AC is not working	1139

(13 rows)

شکل ۱۷ دلایل لغو سفر و تعداد هرکدام به صورت یکپارچه

اما چون براساس نوع خودرو میانگین متفاوتی رو جایگزین می‌کردیم یک کوئری به تفکیک نـوع خـودرو می‌نویسیم:

```sql
SELECT
    vehicle_type,
    ROUND(AVG(driver_rating)::NUMERIC, 2) AS avg_driver,
    ROUND(AVG(customer_rating)::NUMERIC, 2) AS avg_customer,
    COUNT(*) AS trip_count
FROM silver.cleaned_dataset
WHERE booking_status = 'Completed'
GROUP BY vehicle_type
ORDER BY avg_driver DESC;
```

vehicle_type	avg_driver	avg_customer	trip_count
Uber XL	4.24	4.40	2765
Bike	4.23	4.40	13921
eBike	4.23	4.40	6480
Auto	4.23	4.40	22970
Go Sedan	4.23	4.41	16550
Premier Sedan	4.23	4.40	11158
Go Mini	4.23	4.40	18404

(7 rows)

شکل ۱۸ میانگین امتیازها با توجه به نوع خودرو

۶-۲- بررسی تعداد رکوردها و مدیریت داده‌های تکراری

پس از اجرای عملیات ETL و انتقال داده‌ها از لایه برنز به لایه نقره‌ای، متوجه شدم که تعداد رکوردهـای جدول نقره‌ای (silver.cleaned_dataset) با تعداد رکوردهای جدول برنز(bronze.raw_dataset) برابـر نیست. جدول برنز ۱۵۰,۰۰۰ رکورد داشت در حالی که جدول نقره‌ای ۱۴۸,۷۶۷ رکورد، یعنـی **۱,۲۳۳ رکورد کمتر**.

برای پیدا کردن علت، کوئری زیر را اجرا کردم تا ببینم آیا booking_id تکراری وجود دارد یا خیر؟

```
uber_db=# SELECT booking_id, COUNT(*)
uber_db-# FROM bronze.raw_dataset
uber_db-# GROUP BY booking_id
uber_db-# HAVING COUNT(*) > 1;
  booking_id   | count
---------------+-------
 "CNR9555508" |     2
 "CNR9027093" |     2
 "CNR5598384" |     2
 "CNR4042457" |     2
 "CNR2078514" |     2
 "CNR5627941" |     2
 "CNR7096387" |     2
 "CNR5341647" |     2
 "CNR1056023" |     2
 "CNR5358950" |     2
```

شکل ۱۹ بررسی وجود booking_id تکراری

دلیل این اتفاق در انبار داده (Data Warehouse) این است که، ستون booking_id باید به عنوان کلید اصلی (Primary Key) عمل کند و مقدار یکتا داشته باشد. وجود رکوردهای تکراری می‌تواند دلایلی مانند: خطا در فرایند ثبت داده یا ممکن است داده‌ها دو بار از منبع اصلی دریافت شده باشند یا مشکل از منبع داده باشد. رویکرد من در مدیریت این موضوع این است که:

با توجه به اینکه در لایه نقره‌ای می‌خواهیم داده‌هایی تمیز، یکتا و آماده تحلیل داشته باشیم، تصمیم گرفتم از ON CONFLICT (booking_id) DO NOTHING استفاده کنم. این دستور تضمین می‌کند که: فقط اولین نسخه از هر booking_id وارد جدول نقره شود و از ایجاد رکورد تکراری در جدول نقره جلوگیری شود و مهم‌تر از همه کلید اصلی یکتا بماند.

```
uber_db=# SELECT COUNT(*) FROM bronze.raw_dataset;
 count
--------
 150000
(1 row)


uber_db=# SELECT COUNT(*) FROM silver.cleaned_dataset;
 count
--------
 148767
(1 row)
```

شکل ۲۰ مقایسه تعداد رکوردهای جدول Silver و Bronze

گزارش فاز چهارم: Medalion Architecture لایه Gold

اکنون که با موفقیت لایه نقره‌ای رو تکمیل کردیم میرویم به سراغ **لایه طلایی(Gold)** . توی این لایه قراره داده‌ها رو برای مصرف نهایی آماده کنیم: تمیزترین و غنی‌ترین نسخه، به همراه ستون‌های محاسباتی، محدودیت‌های یکپارچگی و کلید اصلی. این لایه مستقیماً در داشبوردها، API و دستیارهای هوشمند Text-To-SQL استفاده خواهدشد. طبق صورت پروژه، سه اقدام اصلی داریم:

- **طراحی Schema نهایی** با نام gold.dataset و انتخاب نوع‌های داده بهینه. از NUMERIC برای دقت مالی و TEXT برای رشته‌ها که انعطاف‌پذیرتر هست استفاده کردم.
- **محاسبه و غنی‌سازی**: اضافه کردن ستون محاسباتی مثل درآمد به ازای هر کیلومتر(revenue_per_km) و...
- **یکپارچگی داده** :تعیین Primary Key و اضافه کردن محدودیت CHECK برای اطمینان از بازه‌ی معتبر امتیازها (مثلاً بین ۰ تا ۵).

علاوه بر این، باید مطمئن بشیم که تمیزترین نسخه داده‌ها رو توی این جدول داریم. از اونجایی که لایه نقره‌ای قبلاً تمیز و غنی شده، کار زیادی برای پاکسازی نمیمونه، اما می‌تونیم مطمئن بشیم که امتیازها در بازه درست هستن و محاسبات به درستی انجام می‌شوند.

مثل قبل اسکریپت رو اینطوری ساختم که همه چیز در یک تراکنش باشه تا اگر مشکلی پیش اومد، هیچ تغییری ذخیره نشه و دیتابیس مشکلی پیدا نکنه.

Insert اصلی رو نوشتم که داده‌ها رو از silver می‌کشه و revenue_per_km رو محاسبه می‌کنه CASE گذاشتم تا بگویم اگر مسافت بیشتر از صفر بود درآمد به ازای کیلومتر حساب بشه، وگرنه null بمونه. (مثل سفرهای لغو شده)، چراکه تقسیم بر صفر خطا می‌ده و منطقی نیست درآمد برای سفر بدون مسافت حساب کنیم:

```
CASE
  WHEN ride_distance > 0 THEN booking_value / ride_distance
  ELSE NULL
END AS revenue_per_km,
```

بقیه ستون‌ها رو مستقیم کپی کردم چون در Silver تمیز شدن، وimputed flags هارو و is_cancelledرو هم نگه داشتم تا در داشبورد بتونیم فیلتر کنیم که کدوم‌ها لغو شدن.

```
ggr@DESKTOP-MO4J70A MINGW64 /d/Daneshgoh/Master/Term 1/Advance Data Base/Tamrin/Project/uber_data_platform/backend/src
$ node gold_etl.js
[dotenv@17.2.4] injecting env (5) from ..\.env -- tip: ◈ encrypt with Dotenvx: https://dotenvx.com
[dotenv@17.2.4] injecting env (0) from ..\..\.env -- tip: ◈ prevent building .env in docker: https://dotenvx.com/prebuild
◈ Connected to PostgreSQL database
◈ Starting Gold Layer ETL ...
◈ Gold table schema created/verified
✅ Data transferred to Gold with revenue_per_km enriched
◈ Gold Layer ETL completed successfully!
```

شکل ۲۱ خروجی موفق عملیات Gold

```
uber_db=# \d gold.gold_dataset;
                    Table "gold.gold_dataset"
          Column           |            Type             | Collation | Nullable | Default
---------------------------+-----------------------------+-----------+----------+---------
 booking_id                | text                        |           | not null |
 trip_timestamp            | timestamp without time zone |           |          |
 pickup_hour               | integer                     |           |          |
 day_name                  | text                        |           |          |
 is_weekend                | boolean                     |           |          |
 booking_status            | text                        |           |          |
 customer_id               | text                        |           |          |
 vehicle_type              | text                        |           |          |
 unified_cancellation_reason | text                      |           |          |
 booking_value             | numeric(10,2)               |           |          |
 ride_distance             | numeric(10,2)               |           |          |
 revenue_per_km            | numeric(10,2)               |           |          |
 driver_rating             | numeric(3,2)                |           |          |
 customer_rating           | numeric(3,2)                |           |          |
 payment_method            | text                        |           |          |
 driver_rating_imputed     | boolean                     |           |          |
 customer_rating_imputed   | boolean                     |           |          |
 is_cancelled              | boolean                     |           |          |
Indexes:
    "gold_dataset_pkey" PRIMARY KEY, btree (booking_id)
Check constraints:
    "gold_dataset_customer_rating_check" CHECK (customer_rating >= 0::numeric AND customer_rating <= 5::numeric)
    "gold_dataset_driver_rating_check" CHECK (driver_rating >= 0::numeric AND driver_rating <= 5::numeric)
```

شکل ۲۲ ساختار و اسکیمای جدول Gold

در پایان ما به اطلاعات مفیدی میانگین درآمد هرسفر به ازای هر کیلومتر:

```
SELECT
  COUNT(*) AS total_rows,
  COUNT(CASE WHEN revenue_per_km IS NULL THEN 1 END) AS null_revenue_per_km,
  ROUND(AVG(revenue_per_km), 2) AS avg_revenue_per_km
FROM gold.gold_dataset;
```

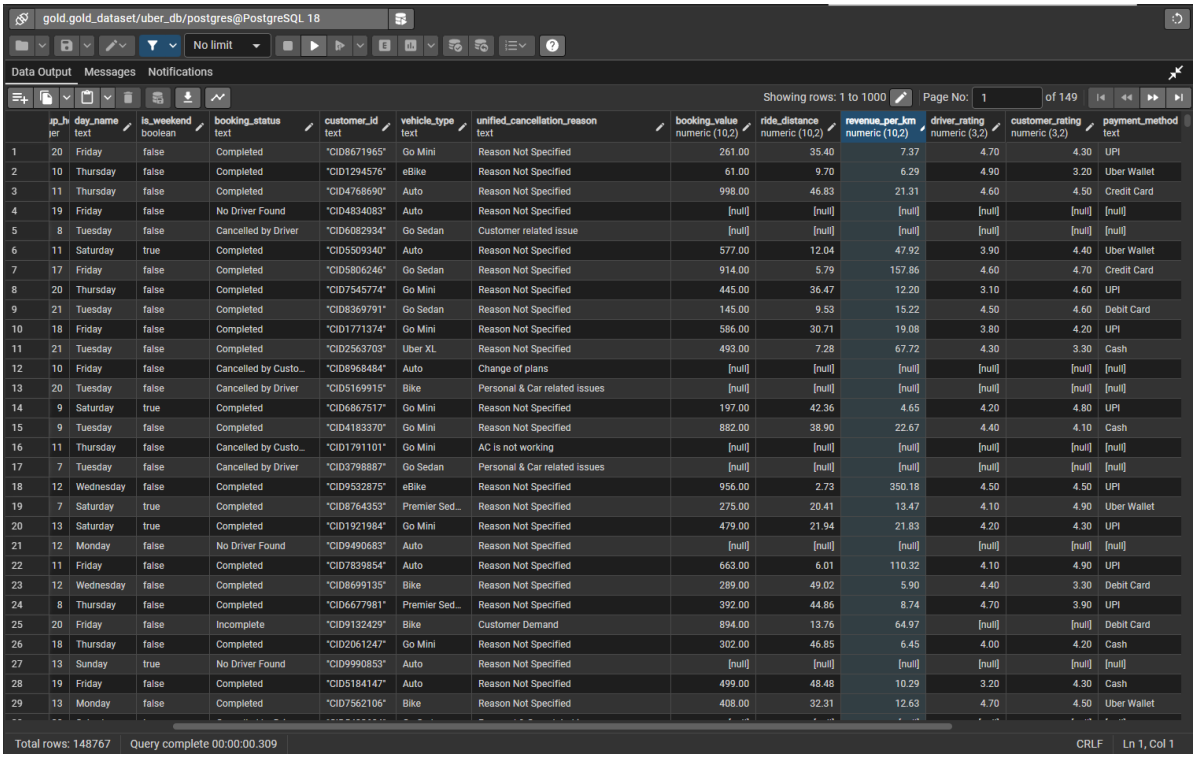
```
uber_db=# SELECT
uber_db-#   COUNT(*) AS total_rows,
uber_db-#   COUNT(CASE WHEN revenue_per_km IS NULL THEN 1 END) AS null_revenue_per_km,
uber_db-#   ROUND(AVG(revenue_per_km), 2) AS avg_revenue_per_km
uber_db-# FROM gold.gold_dataset;
 total_rows | null_revenue_per_km | avg_revenue_per_km
------------+---------------------+--------------------
     148767 |               47592 |              37.83
(1 row)
```

میانگین درامد به تفکیک وسیله نقلیه:

```
SELECT
  vehicle_type,
  COUNT(*) AS ride_count,
  ROUND(AVG(revenue_per_km), 2) AS avg_revenue_per_km
FROM gold.gold_dataset
WHERE revenue_per_km IS NOT NULL
GROUP BY vehicle_type
ORDER BY avg_revenue_per_km DESC;
```

```
 vehicle_type  | ride_count | avg_revenue_per_km
---------------+------------+--------------------
 Go Sedan      |      18181 |              38.23
 Bike          |      15239 |              38.14
 Auto          |      25218 |              38.08
 Uber XL       |       3023 |              37.69
 Premier Sedan |      12208 |              37.62
 Go Mini       |      20198 |              37.61
 eBike         |       7108 |              36.33
(7 rows)
```

شکل ۲۳ میانگین درامد در هر کلیومتر به تفکیک نوع وسیله نقلیه



	...per text	day_name text	is_weekend boolean	booking_status text	customer_id text	vehicle_type text	unified_cancellation_reason text	booking_value numeric (10,2)	ride_distance numeric (10,2)	revenue_per_km numeric (10,2)	driver_rating numeric (3,2)	customer_rating numeric (3,2)	payment_method text
1	20	Friday	false	Completed	"CID8671965"	Go Mini	Reason Not Specified	261.00	35.40	7.37	4.70	4.30	UPI
2	10	Thursday	false	Completed	"CID1294576"	eBike	Reason Not Specified	61.00	9.70	6.29	4.90	3.20	Uber Wallet
3	11	Thursday	false	Completed	"CID4768690"	Auto	Reason Not Specified	998.00	46.83	21.31	4.60	4.50	Credit Card
4	19	Friday	false	No Driver Found	"CID4834083"	Auto	Reason Not Specified	[null]	[null]	[null]	[null]	[null]	[null]
5	8	Tuesday	false	Cancelled by Driver	"CID6082934"	Go Sedan	Customer related issue	[null]	[null]	[null]	[null]	[null]	[null]
6	11	Saturday	true	Completed	"CID5509340"	Auto	Reason Not Specified	577.00	12.04	47.92	3.90	4.40	Uber Wallet
7	17	Friday	false	Completed	"CID5806246"	Go Sedan	Reason Not Specified	914.00	5.79	157.86	4.60	4.70	Credit Card
8	20	Thursday	false	Completed	"CID7545774"	Go Mini	Reason Not Specified	445.00	36.47	12.20	3.10	4.60	UPI
9	21	Tuesday	false	Completed	"CID8369791"	Go Sedan	Reason Not Specified	145.00	9.53	15.22	4.50	4.60	Debit Card
10	18	Friday	false	Completed	"CID1771374"	Go Mini	Reason Not Specified	586.00	30.71	19.08	3.80	4.20	UPI
11	21	Tuesday	false	Completed	"CID2563703"	Uber XL	Reason Not Specified	493.00	7.28	67.72	4.30	3.30	Cash
12	10	Friday	false	Cancelled by Custo...	"CID8968484"	Auto	Change of plans	[null]	[null]	[null]	[null]	[null]	[null]
13	20	Tuesday	false	Cancelled by Driver	"CID5169915"	Bike	Personal & Car related issues	[null]	[null]	[null]	[null]	[null]	[null]
14	9	Saturday	true	Completed	"CID6867517"	Go Mini	Reason Not Specified	197.00	42.36	4.65	4.20	4.80	UPI
15	9	Tuesday	false	Completed	"CID4183370"	Go Mini	Reason Not Specified	882.00	38.90	22.67	4.40	4.10	Cash
16	11	Thursday	false	Cancelled by Custo...	"CID1791101"	Go Mini	AC is not working	[null]	[null]	[null]	[null]	[null]	[null]
17	7	Tuesday	false	Cancelled by Driver	"CID3798887"	Go Sedan	Personal & Car related issues	[null]	[null]	[null]	[null]	[null]	[null]
18	12	Wednesday	false	Completed	"CID9532875"	eBike	Reason Not Specified	956.00	2.73	350.18	4.50	4.50	UPI
19	7	Saturday	true	Completed	"CID8764353"	Premier Sed...	Reason Not Specified	275.00	20.41	13.47	4.10	4.90	Uber Wallet
20	13	Saturday	true	Completed	"CID1921984"	Go Mini	Reason Not Specified	479.00	21.94	21.83	4.20	4.30	UPI
21	12	Monday	false	No Driver Found	"CID9490683"	Auto	Reason Not Specified	[null]	[null]	[null]	[null]	[null]	[null]
22	11	Friday	false	Completed	"CID7839854"	Auto	Reason Not Specified	663.00	6.01	110.32	4.10	4.90	UPI
23	12	Wednesday	false	Completed	"CID8699135"	Bike	Reason Not Specified	289.00	49.02	5.90	4.40	3.30	Debit Card
24	8	Thursday	false	Completed	"CID6677981"	Premier Sed...	Reason Not Specified	392.00	44.86	8.74	4.70	3.90	UPI
25	20	Friday	false	Incomplete	"CID9132429"	Bike	Customer Demand	894.00	13.76	64.97	[null]	[null]	Debit Card
26	18	Thursday	false	Completed	"CID2061247"	Go Mini	Reason Not Specified	302.00	46.85	6.45	4.00	4.20	Cash
27	13	Sunday	true	No Driver Found	"CID9990853"	Auto	Reason Not Specified	[null]	[null]	[null]	[null]	[null]	[null]
28	19	Friday	false	Completed	"CID5184147"	Auto	Reason Not Specified	499.00	48.48	10.29	3.20	4.30	Cash
29	13	Monday	false	Completed	"CID7562106"	Bike	Reason Not Specified	408.00	32.31	12.63	4.70	4.50	Uber Wallet

شکل ۲۴ جدول Gold

در نهایت یک مقایسه کوتاهی از سه لایه داشته باشیم:

جدول2 مقایسه سه لایه معماری مدالیون

ویژگی	*Bronze (Raw)*	*Silver (Cleaned)*	*Gold (Business Ready)*
تعداد ستون	۱۷	۲۱ (با ویژگی‌های جدید)	۱۸ (بهینه‌شده)
revenue_per_km	ندارد	ندارد	دارد (با مدیریت صفر)
unified_cancellation_reason	ندارد	دارد	دارد (بهینه)
Primary Key	ندارد	دارد	دارد
Nullدر revenue_per_km	—	—	فقط در سفرهای لغو یا ناقص
آماده برای داشبورد	خیر	نیمه‌آماده	کاملاً آماده

گزارش فاز پنجم: پیاده‌سازی CRUD API

در این بخش ما به پیاده‌سازی بک‌اند (backend) به کمک express.js و محیط توسعه Node.js پرداختیم و سپس با استفاده از REST-full API's و ساخت endpoint های مختلف برای فرانت‌اند توانستیم اطلاعات ذخیره شده داخل دیتابیس را در یک داشبورد عملیاتی به صورت ویژوال و با UI/UX مناسب نمایش بدهیم. (این در بخش بعد به طول کامل بررسی می‌شود).

۱-۲- ساختار تعریف شده Backend:

توی یه پروژه بک‌اند باExpress ، ما کد رو به بخش‌های مختلف تقسیم می‌کنیم تا تمیز و قابل فهم باشه:

- **routes (مسیرها):** اینجا مشخص می‌کنیم که برای هر آدرس (URL) و متد (GET, POST, ...) کد کجا باید اجرا بشه. مثلاً اگه کاربر به /api/trips درخواستGET بده، باید بره تابع getTrips رو اجرا کنه
- **controllers (کنترلرها):** اینجا توابع اصلی رو می‌نویسیم که کار واقعی رو انجام می‌دن (مثل کوئری زدن به دیتابیس، پردازش داده، برگرداندن نتیجه و ...) - tripCotroller در برنامه این کار رو انجام می‌دهد.
- **models:** اینجا معمولاً کدهایی برای ارتباط با دیتابیس می‌ذاریم (مثل اتصالpool) - db.js در برنامه این کار رو انجام می‌دهد.

این جداسازی باعث میشه اگه بعداً خواستیم یه چیزی رو عوض کنیم، راحتتر باشه. مثلاً اگه مسیر یه APIعوض بشه، فقط فایل routes رو تغییر می‌دیم و کاری به کنترلر نداریم

برای پیاده‌سازی عملی و اصولی ما ۳ مرحله داریم:

۱. نصب dependency های لازم: که بعدا در فایل packages.json می‌بینیم.

۲. ایجاد اتصال بین دیتابیس و برنامه:

```
const { Pool } = require("pg");
require("dotenv").config({ path: "../../.env" });
const pool = new Pool({
  user: process.env.DB_USER,
  host: process.env.DB_HOST,
  database: process.env.DB_NAME,
  password: process.env.DB_PASSWORD,
  port: process.env.DB_PORT,
});
pool.on("connect", () => {
  console.log("🔗 Connected to PostgreSQL database");
});
pool.on("error", (err) => {
  console.error("❌ Unexpected error on idle client", err);
  process.exit(-1);
});
module.exports = pool
```

۳. ساخت سرور بک اند (run server) و مدیریت route ها:

```
require('dotenv').config({ path: '../.env' });
const express = require('express');
const cors = require('cors');
const app = express();
// Middleware for read request
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(cors());                          // beacuse we have seprated frontend
const tripRoutes = require('./routes/trips');
app.use('/api/trips', tripRoutes);
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

۴. پیاده‌سازی عملیات CRUD در کنترلرها و تست از طریق ابزار (در اینجا از postman استفاده شده)

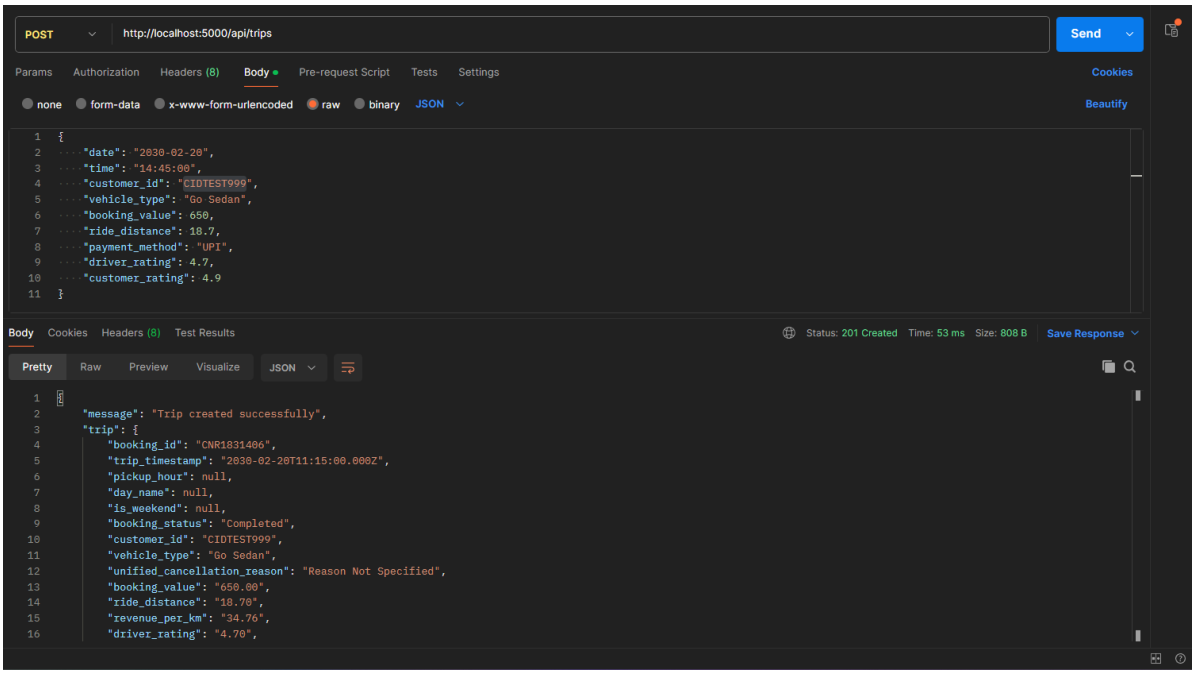
۲-۲- عملیات CRUD

پس از پیاده‌سازی فانکشن‌ها و عملیات‌ها ما توسط ۴ انـدپوینت، عملکردهـای خواسـته شـده در صـورت پروژه رو برآورده می‌کنیم:

```
const tripController = require("../controllers/tripController");

router.post("/", tripController.createTrip);
router.get("/", tripController.getTrips);
router.patch("/:id", tripController.updateTripStatus);
router.delete("/:id", tripController.deleteTrip);
```

۱-۲-۳- CREATE - Post API



شکل ۲۵ Create - ساخت یک سفر جدید

در اینجا همانطور که مشاهده می‌کنیم از طریق روت /api/trips و متد POST، ما اطلاعات ضرروی تعریف شده رو از طریق body و به صورت JSON به این روت می‌فرستیم و این اطلاعات تحت عنوان یک سفر جدید داخل پایگاه داده با موفقیت ذخیره می‌شوند.

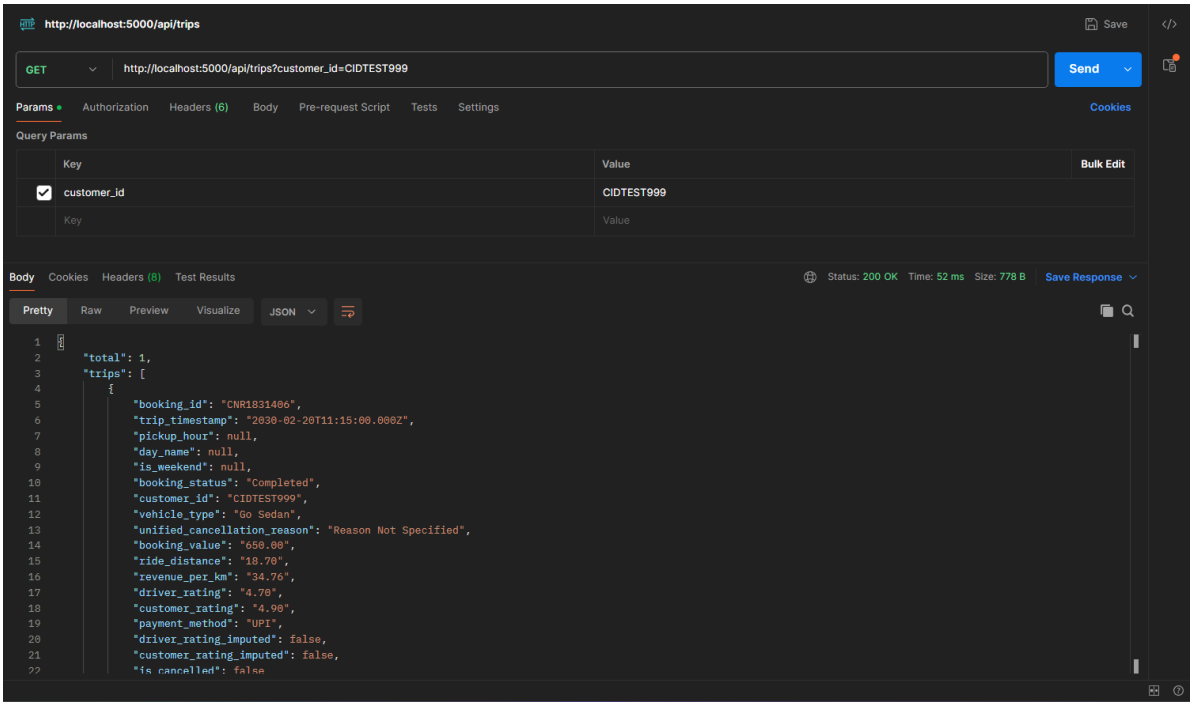
فرض بر این بوده که وضعیت سفر Completed و آیدی سفر یا همان booking_id به صورت خودکار تولید شود. بقیه اطلاعات مانند اطلاعات تاریخ، زمان، نوع خودرو و روش پرداخت و امتیازها از کاربر به صورت Optional دریافت شود.

۲-۳-۲ READ - GET API

در اینجا از طریق متد GET و از طریق روت api/trips/ می‌توانیم به دو صورت عمل کنیم:

۱. همه رکوردها رو برگردانیم.

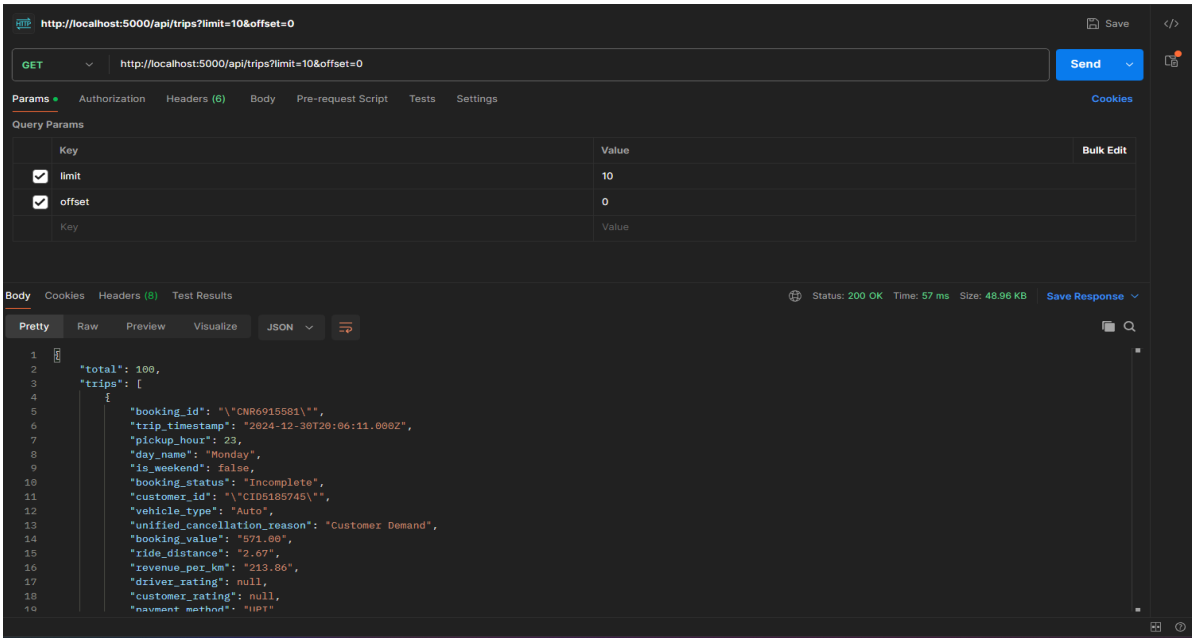
۲. یک رکورد خاص، رو بتوانیم از طریق customer_id یعنی شناسه مشتری برگردانیم.



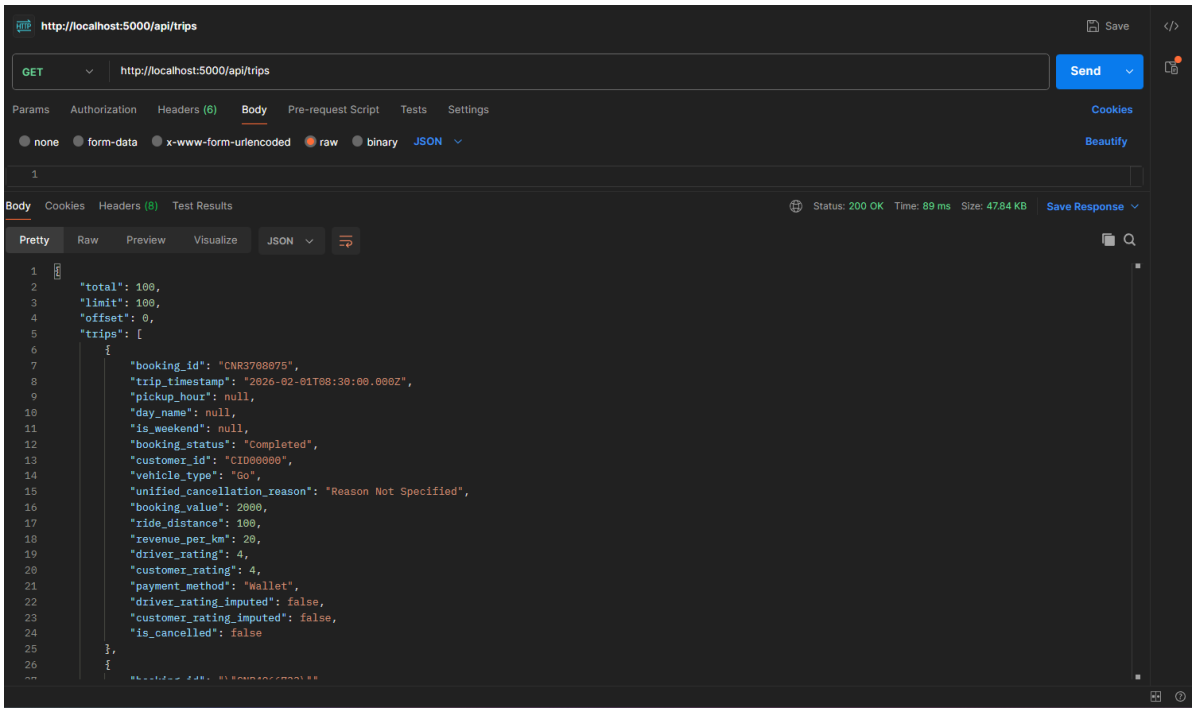
شکل ۲۶ READ - خواندن یک رکورد با customer_id مشخص

کافی است customer_id رو از طریق Query Params دریافت کنیم و داخل پایگاه‌داده سرچ بزنیم. همانطور که مشاهده می‌شود رکورد مورد نظر پیدا شده است.

حالتی که بتوانیم تمامی رکوردها رو بدون هیچ شرطی بخوانیم هم محیا هست اما برای کنترل بیشتر می‌توانیم داخل Query Params لیمیت و آفست تنظیم کنیم:



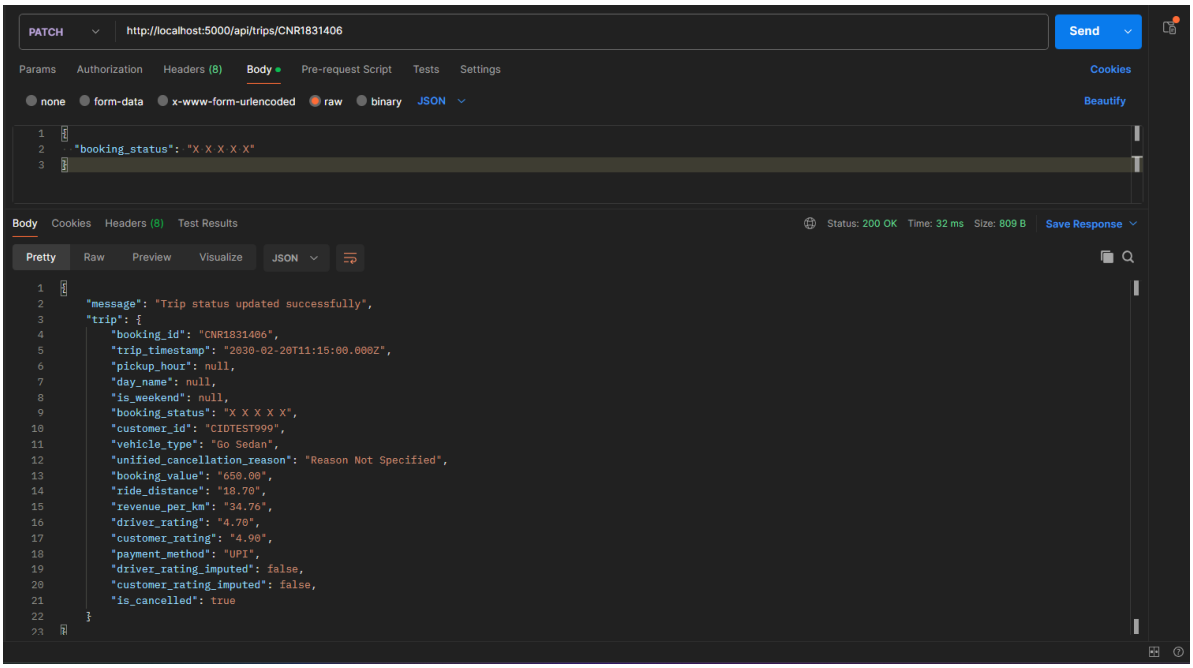
شکل ۲۷ READ–خواندن همه رکوردها یا اعمال فیلتر با کمک limit و offset



شکل ۲۸ READ - خواندن همه رکوردها بدون هیچ فیلتری

۳-۲-۳ UPDATE - PATCH API

برای بروزرسانی یک سفر هم کافی است آیدی آن رزرو رو در URL بفرستیم و همچنین وضعیت مورد نظرمان در مورد آن سفر رو هم در Body ارسال کنیم تا بروزرسانی ما انجام شود:



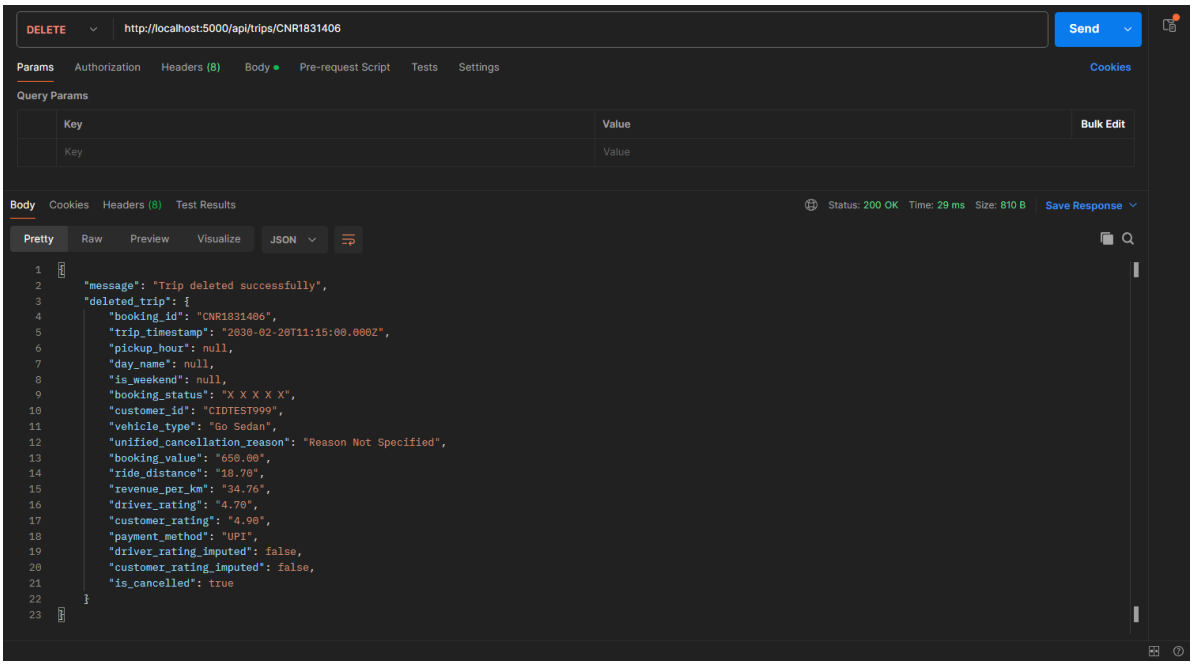
شکل ۲۹ UPDATE - بروزرسانی وضعیت یک سفر از طریق booking_id

همانطور که مشاهده می‌شود: ما وضعیت سفر رو با کمک آیدی: CNR1831406 به مقدار دلخواه:

“X X X X X X” : Booking_status تغییر داده‌ایم.

۴-۳-۲- DELETE API

ما از همین آیدی CNR1831406 استفاده می‌کنیم و این سفر رزرو شده رو حذف می‌کنیم:



شکل ۳۰ DELETE - حذف یک سفر

گزارش فاز ششم : داشبورد تحلیلی و تعاملی

در این بخش هدف من این بود که داده‌های تمیز و مدل‌سازی‌شده‌ی لایه Gold را به یک داشبورد تحلیلی تبدیل کنم که هم از نظر بصری حرفه‌ای باشد و هم از نظر منطقی، پاسخ‌گوی نیازهای تحلیلی پروژه باشد. تمرکز اصلی من این بود که تمام شاخص‌ها و نمودارها به صورت داینامیک و وابسته به فیلترها کار کنند و صرفاً یک صفحه‌ی نمایشی ساده نباشد.

۱-۲- طراحی معماری سمت بک‌اند برای داشبورد

برای اینکه داشبورد واقعاً تحلیلی باشد، تصمیم گرفتم منطق تجمیع (Aggregation) را در سمت دیتابیس انجام دهم، نه در فرانت‌اند. دلیل این تصمیم این بود که دیتابیس برای عملیات GROUP BY و SUM و COUNT بهینه‌تر است همچنین از انتقال حجم زیاد داده‌ی خام به فرانت جلوگیری می‌شود و ساختار پروژه حرفه‌ای‌تر و مقیاس‌پذیرتر می‌شود.

برای این منظور، چند endpoint جداگانه برای داشبورد طراحی کردم:

- دریافت شاخص‌های کلیدی عملکرد(KPI[1]s)
- دریافت داده‌های نمودارها
- دریافت لیست سفرها با pagination و فیلتر
- پشتیبانی کامل از فیلتر بازه زمانی و نوع خودرو

تمام این API ها از پارامترهای زیر پشتیبانی می‌کنند:

- startDate
- endDate
- vehicleType

[1] Key Performance Indicator : شاخص کلیدی عملکرد

به این صورت، هر بار که کاربر فیلتر را تغییر می‌دهد، داده‌های داشبورد بـه صـورت کامـل به‌روزرسـانی می‌شوند. تقریباً در تمام توابع تحلیلی، یک الگوی مشترک رعایت شده است:

ابتدا فیلترها از req.query دریافت می‌شوند:

```
const { startDate, endDate, vehicleType } = req.query;
```

سپس سه متغیر برای ساخت شرط‌های پویا تعریف می‌شود:

```
let whereClauses = [];
let values = [];
let idx = 1;
```

- whereClauses: برای نگهداری شرط‌ها
- values: برای مقادیر پارامتری
- idx: برای شماره‌گذاری پارامترها($1, $2, ...)

به عنوان مثال اگر کاربر تاریخ شروع را ارسال کرده باشد، شرط زیر اضافه می‌شود:

```
trip_timestamp >= $1
```

و مقدار آن در آرایه values ذخیره می‌شود. این روش باعث می‌شود کوئری‌ها به صورت Parameterized اجرا شوند و از SQL Injection جلوگیری شود.

در نهایت اگر شرطی وجود داشته باشد، بخش WHERE به صورت پویا ساخته می‌شود و به کوئری اصلی اضافه می‌گردد.

```
const whereSQL = whereClauses.length > 0 ? `WHERE ${whereClauses.join(' AND ')}` : '';
```

۲-۲- مدیریت ایمن مقادیر عددی

از آنجا که خروجی‌های PostgreSQL ممکن است به صورت رشته یا مقدار null برگردند، تابعی بـه نـام safeNumber تعریف شد. این تابع ابتدا بررسی می‌کند که مقدار null نباشد، سپس آن را به عدد تبدیل می‌کند و در صورتی که تبدیل نامعتبر باشد (NaN شود)، مقدار پیش‌فرض برمی‌گرداند. بـه ایـن ترتیـب داده‌های ارسال‌شده به فرانت‌اند همیشـه عـددی و قابـل اسـتفاده در نمودارهـا هسـتند و از بـروز خطـا جلوگیری می‌شود.

```
// Helper function to safely parse numeric values
const safeNumber = (value, fallback = null) => {
  if (value == null) return fallback;
  const num = Number(value);
  return Number.isNaN(num) ? fallback : num;
};
```

شکل ۳۱ کد تابع safeNumber برای مدیریت مقادیر عددی

۳-۲- شاخص‌های کلیدی عملکرد (KPI)

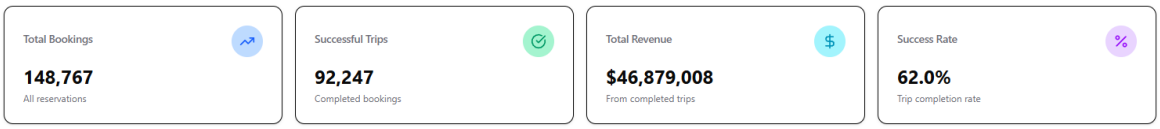
در این قسمت، چهار شاخص اصلی پیاده‌سازی شد:

- تعداد کل رزروها
- تعداد رزروهای موفق
- مجموع درآمد
- نرخ موفقیت سفر

منطق این بخش به صورت مستقیم داخل یک کوئری تجمیعی در PostgreSQL نوشته شد. به طور خلاصه، از COUNT و SUM و FILTER استفاده کردم تا بتوانم همزمان چند شاخص را از یک اسکن دیتاست استخراج کنم.

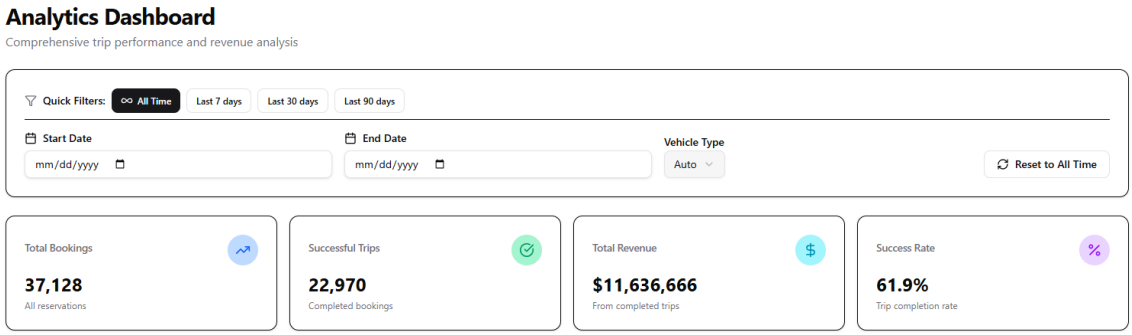
نمونه‌ای از منطق استفاده‌شده:

```
const query = `
    SELECT
      COUNT(*) AS total_bookings,
      COUNT(*) FILTER (WHERE is_cancelled = FALSE) AS successful_bookings,
      SUM(booking_value) FILTER (WHERE is_cancelled = FALSE) AS total_revenue,
      ROUND(
        (COUNT(*) FILTER (WHERE is_cancelled = FALSE)::NUMERIC / NULLIF(COUNT(*), 0)) * 100,2) AS
        success_rate
    FROM gold.gold_dataset
    ${whereSQL}
  `;
```



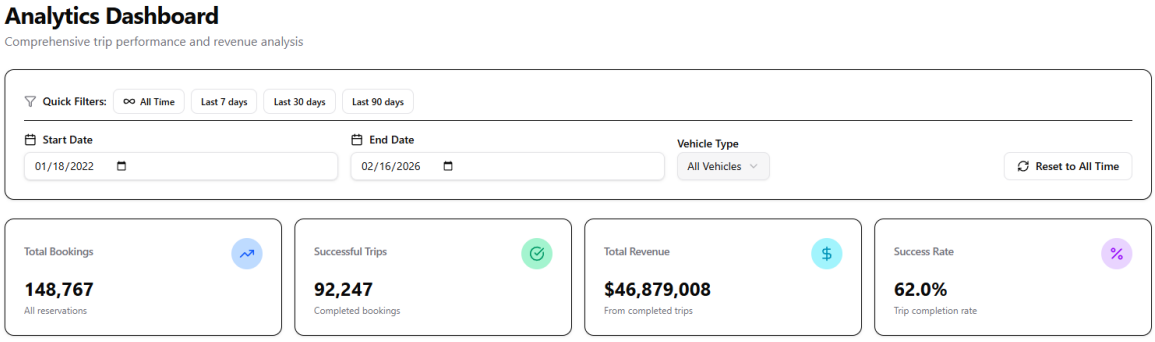
شکل ۳۲ نمایش KPI ها در داشبورد

همچنین ما می‌توانیم با تعیین پارامترها و فیلترهای اضافه مانند فیلترزمانی مثل: تاریخ شروع و تاریخ پایان یا فیلتر بر اساس نوع خودرو، KPI های مربوطه رو خیلی سریع مشاهده کنیم:



شکل ۳۳ نمایش KPI ها با فیلتر نوع خودرو

همچنین فیلترهای آماده‌ای مانند ۷روز اخیر، یک ماه اخیر و ۹۰ روز اخیر به صورت خیلی سریع دردسترس قرار دارند.



شکل ۳۴ نمایش KPI ها با فیلتر زمانی

۲-۴- نمودارها(Charts)

در این بخش، برای هر نمودار یک Endpoint مجزا در بک‌اند پیاده‌سازی شد که داده‌ها را از جدول gold.gold_dataset استخراج کرده و پس از تجمیع، به فرانت‌اند ارسال می‌کند.

تمام نمودارها قابلیت اعمال فیلتر بازه زمانی (startDate, endDate) و نوع خودرو (vehicleType) را دارند که به صورت پویا در بخش WHERE کوئری اضافه می‌شوند.

۲-۳-۱- نمودار دایره‌ای توزیع دلایل لغو

برای تحلیل علل شکست سفرها، از ستون unified_cancellation_reason استفاده کردم. در ابتدای تابع مربوطه (getCancellationReasons) شرط زیر به صورت پیش‌فرض در WHERE قرار داده شده است:

```
let whereClauses = ['is_cancelled = TRUE'];
```

یعنی فقط سفرهای لغوشده بررسی می‌شوند. سپس کوئری اصلی به شکل زیر نوشته شده است:

```
const query = `
  SELECT
    unified_cancellation_reason AS reason,
    COUNT(*) AS count,
    ROUND((COUNT(*)::NUMERIC / SUM(COUNT(*)) OVER ()) * 100, 2) AS percentage
  FROM gold.gold_dataset
  ${whereSQL}
  GROUP BY unified_cancellation_reason
  ORDER BY count DESC
`;
```

شکل ۳۵ کد مربوط به سفرهای لغو شده

در این کوئری:

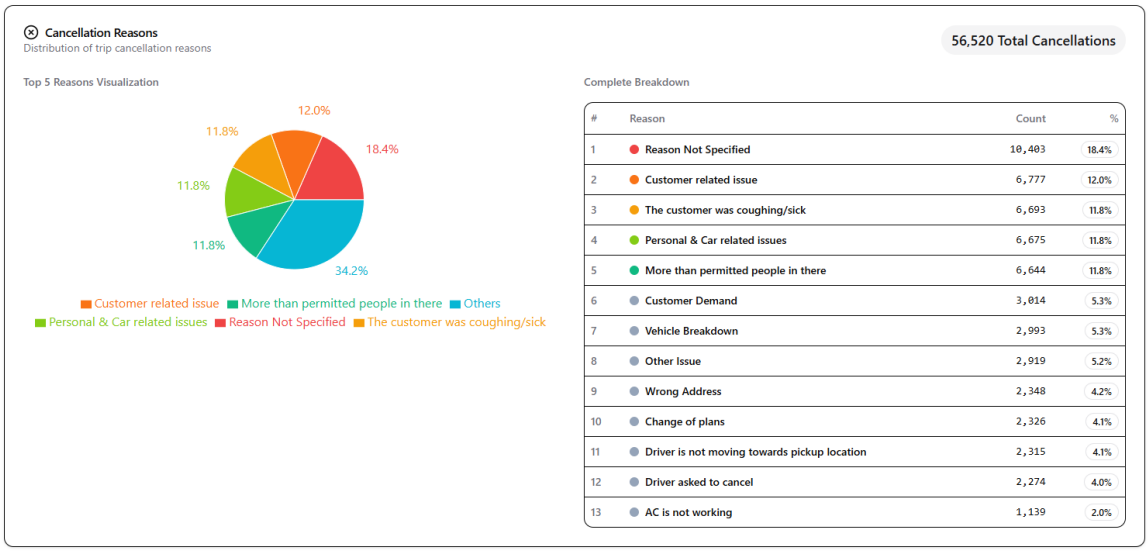
COUNT(*) تعداد تکرار هر دلیل لغو را محاسبه می‌کند. سپس با استفاده از

GROUP BY unified_cancellation_reason داده‌ها بر اساس دلیل لغو دسته‌بندی می‌شوند.

و برای محاسبه درصد هر دلیل از کل لغوها، از Window Function استفاده شده است:

```
SUM(COUNT(*)) OVER ()) * 100, 2)
```

این عبارت مجموع کل لغوها را محاسبه می‌کند و سپس تعداد هر گروه بر آن تقسیم می‌شود تا درصد به دست آید. در نهایت نتایج بر اساس بیشترین تعداد مرتب شده‌اند تا مهم‌ترین دلایل در ابتدای نمودار نمایش داده شوند.



شکل ۳۶ نمودار دلایل لغو سفر

در فرانت‌اند نحوه نمایش حتی بهتر هم شده و درقسمت راست کلیه دلایل لغو سفرها رو بر اساس درصد نمایش داده‌ایم و درسمت چپ با نمودار دایره‌ای ابتدا ۵ نتیجه برتر (بیشترین دلایل لغو سفر) رو نمایش داده و بقیه رو (برای اینکه شلوغ نشود) در قسمت others قرار دادیم. این شکلی خیلی به تحلیل ما کمک بیشتری می‌کند. کمک می‌کند مشخص شود بیشترین علت شکست سفرها چه بوده و تمرکز بهبود سیستم باید روی کدام بخش باشد.

۲-۳-۲- نمودار توزیع روش‌های پرداخت

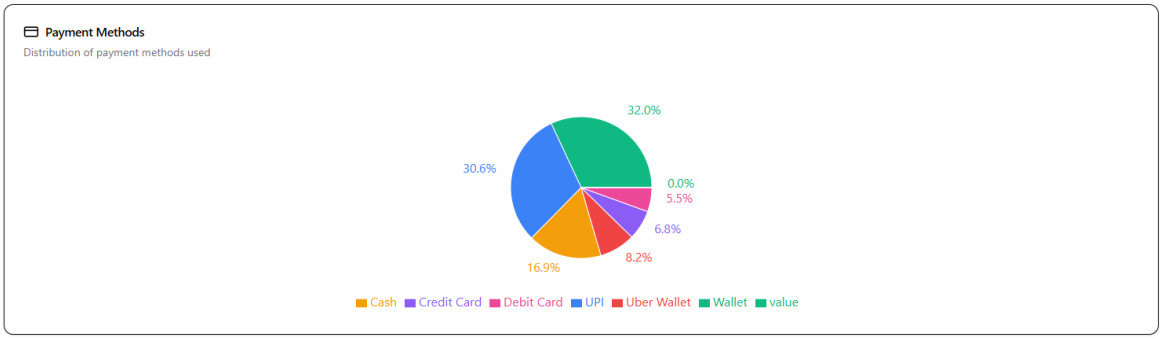
در این بخش، تحلیل روش‌های پرداخت انجام شده است. کوئری مربوط به تابع getPaymentMethods به صورت زیر است:

```
const query = `
      SELECT
        payment_method,
        COUNT(*) AS count,
        ROUND((COUNT(*)::NUMERIC / SUM(COUNT(*)) OVER ()) * 100, 2) AS percentage
      FROM gold.gold_dataset
      ${whereSQL}
      GROUP BY payment_method
      ORDER BY count DESC
    `;
```

در این کوئری:

داده‌ها بر اساس payment_methodگروه‌بندی شده‌اند.

COUNT(*) تعداد استفاده از هر روش پرداخت را محاسبه می‌کند، درصد هر روش نسبت به کل سفرها با همان منطق Window Function محاسبه شده است و نتایج بر اساس بیشترین استفاده مرتب شده‌اند.



شکل ۳۷ نمودار دایره‌ای روش‌های پرداخت

این نمودار نشان می‌دهد چه درصدی از کاربران از هر روش پرداخت استفاده کرده‌اند و برای تحلیل رفتار مالی بسیار مفید است، زیرا می‌توان تشخیص داد کدام روش محبوب‌تر است و آیا نیاز به توسعه یا بهبود روش خاصی وجود دارد یا خیر.

۳-۳-۲- نمودار مقایسه نوع خودرو

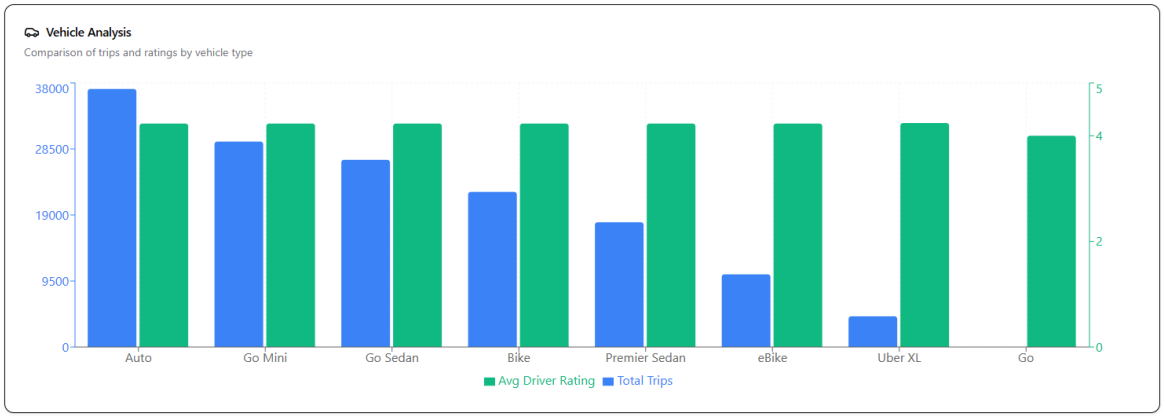
در این بخش، برای هر نوع خودرو موارد زیر محاسبه شدند:

- تعداد سفرهای موفق
- میانگین امتیاز راننده
- میانگین امتیاز مشتری
- مجموع درآمد تولیدشده
- میانگین ارزش هر رزرو

کوئری مربوط به تابع getVehicleAnalysis به صورت زیر است:

```
const query = `
      SELECT
        vehicle_type,
        COUNT(*) AS total_trips,
        ROUND(AVG(driver_rating), 2) AS avg_driver_rating,
        ROUND(AVG(customer_rating), 2) AS avg_customer_rating,
        SUM(booking_value) AS total_revenue,
        ROUND(AVG(booking_value), 2) AS avg_booking_value
      FROM gold.gold_dataset
      ${whereSQL}
      GROUP BY vehicle_type
      ORDER BY total_trips DESC
    `;
```

میانگین‌ها با استفاده از ROUND تا دو رقم اعشار گرد شده‌اند و داده‌ها بر اساس بیشترین تعداد سفر مرتب شده‌اند تا مشخص شود کدام نوع خودرو بیشترین استفاده را داشته است.



شکل ۳۸ نمودار میله‌ای مقایسه انواع خودرو

این بخش امکان تحلیل عملکرد هر کلاس خودرو را فراهم می‌کند و مشخص می‌کند کدام دسته بیشترین رضایت یا بیشترین استفاده را داشته است. همچنین می‌توان از آن برای تصمیم‌گیری در تخصیص منابع استفاده کرد.

۴-۳-۲- نمودار ساعات پرتردد

برای تحلیل الگوی زمانی استفاده از سیستم، از ستون زمان شروع سفر (request_datetime) استفاده شد. هدف این نمودار شناسایی ساعات پرتردد شبانه‌روز است.

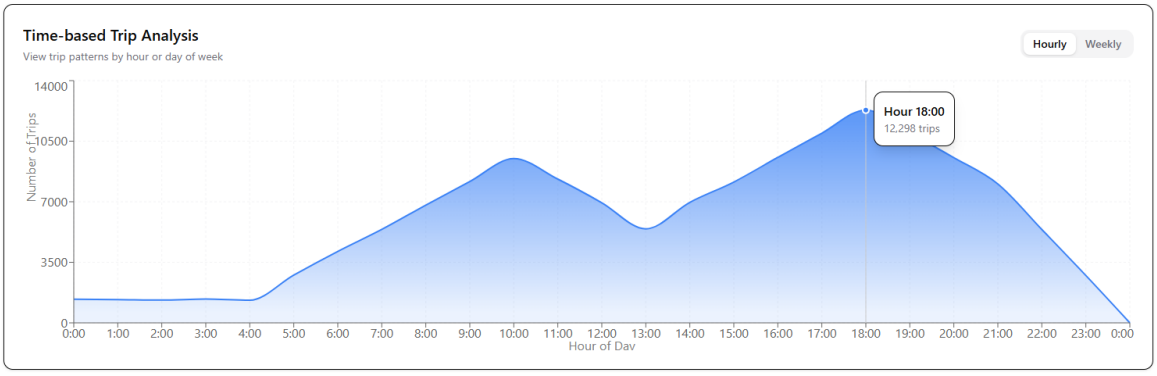
در این بخش، ساعت از فیلد تاریخ با استفاده از تابع EXTRACT(HOUR FROM ...) استخراج شده است.کوئری مربوط به تابع getPeakHours به صورت زیر است:

```
const query = `
      SELECT
        pickup_hour AS hour,
        COUNT(*) AS trip_count
      FROM gold.gold_dataset
      ${whereSQL}
      GROUP BY pickup_hour
      ORDER BY pickup_hour
    `;
```

در این کوئری:

با استفاده از EXTRACT(HOUR FROM request_datetime) ساعت هر سفر استخراج می‌شود سپس COUNT(*) ، تعداد سفرهای ثبت‌شده در هر ساعت را محاسبه می‌کند. و در نهایت داده‌ها بر

اساس ساعت به صورت صعودی مرتب می‌شوند تا نمودار خطی یا ستونی ترتیب طبیعی زمانی داشته باشد.



شکل ۳۹ نمودار خطی ساعات پرتردد

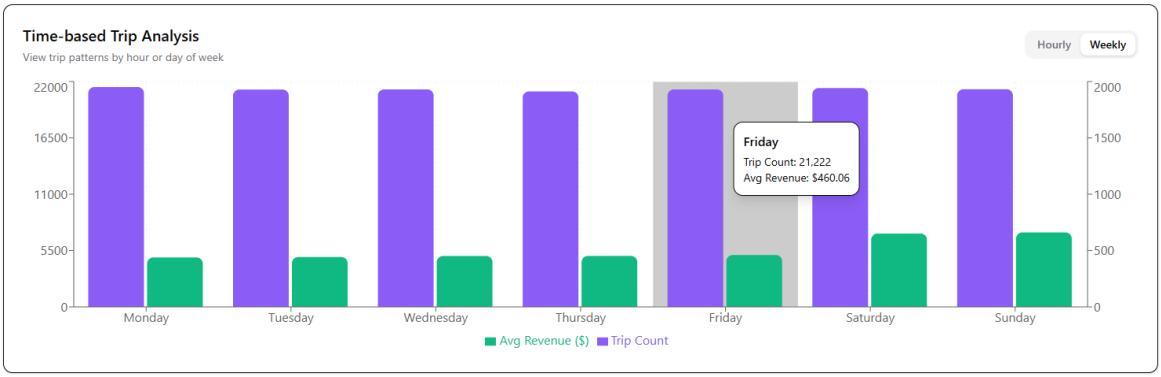
این نمودار نشان می‌دهد بیشترین حجم سفر در چه ساعاتی از شبانه‌روز اتفاق می‌افتد.

۵-۳-۲- نمودار تحلیل روزهای هفته

برای تحلیل رفتار کاربران در طول هفته، روز هفته از تاریخ سفر استخراج شده است

کوئری مربوط به تابع getWeekdayAnalysis به صورت زیر است:

```
const query = `
    SELECT
      day_name,
      COUNT(*) AS trip_count,
      ROUND(AVG(booking_value), 2) AS avg_revenue
    FROM gold.gold_dataset
    ${whereSQL}
    GROUP BY day_name
    ORDER BY
      CASE day_name
        WHEN 'Monday' THEN 1
        WHEN 'Tuesday' THEN 2
        WHEN 'Wednesday' THEN 3
        WHEN 'Thursday' THEN 4
        WHEN 'Friday' THEN 5
        WHEN 'Saturday' THEN 6
        WHEN 'Sunday' THEN 7
      END
  `;
```



شکل ۴۰ نمودار میله‌ای تعداد سفرها براساس روز هفته

همانطور که به صورت بصری هم دیده می‌شود سفر ها تقریبا به اندازه یکسان در طول هفته انجام شده اند (البته این سفرها در بازه all-time یا کل داده‌ها) محاسبه شده‌اند. اما همانطور که دیده می‌شود به طور مشخص میانگین درآمد در روز های Saturday و Sunday یعنی شنبه - یکشنبه بیشتر بوده است. مانند بقیه نمودار ها این نمودار نیز با هاور کردن روی هر قسمت اطلاعات کامل آن نمایش داده می‌شود.

گزارش فاز هفتم: دستیار هوشمند Text to SQL

خب در این بخش ما باید امکاناتی رو فراهم می‌کردیم که مدیر یا هر کاربری بتونه به زبان ساده انگلیسی سوال بپرسه مثلا What is the average rating by vehicle type? و سیستم خودش یک کوئری SQL درست و امن بسازه و نشون بده. مهم‌ترین چیز این بود که سیستم فقط SELECT بزنه و هیچ‌وقت داده رو تغییر نده یا کوئری خطرناک ننویسه.

۱-۲- پیاده‌سازی بک‌اند

من اول یه فایل جداگانه به اسم sqlAssistant.js توی routes ساختم و یه اندپوینت POST به اسم /api/sql-assistant تعریف کردم. وقتی کاربر سوالش رو می‌فرسته، سیستم اول چک می‌کنه که سوال خالی نباشه و طولش بیشتر از ۵۰۰ کاراکتر نباشه (برای جلوگیری از درخواست‌های سنگین). بعد یه پرامپت خیلی مهم به مدل می‌فرستم که شامل کل schema جدول gold.gold_dataset هست. این schema رو توی یه ثابت به اسم GOLD_SCHEMA نگه داشتم که همه ستون‌ها، نوع داده‌ها و توضیحات مهم رو دقیق نوشته.

```
const GOLD_SCHEMA = `
Database Schema - Gold Layer (gold.gold_dataset):
Table: gold.gold_dataset
Columns:
- booking_id (TEXT, PRIMARY KEY): Unique booking identifier (e.g., 'CNR1234567')
- trip_timestamp (TIMESTAMP): Date and time of the trip
- pickup_hour (INTEGER): Hour of day (0-23)
- day_name (TEXT): Day of week name (e.g., 'Monday', 'Tuesday')
- is_weekend (BOOLEAN): Whether the trip occurred on weekend
- booking_status (TEXT): Status of booking (e.g., 'Completed', 'Cancelled')
- customer_id (TEXT): Customer identifier
- vehicle_type (TEXT): Type of vehicle (e.g., 'Sedan', 'Auto', 'eBike', 'Bike', 'Moto', 'Mini', 'XL', 'Go')
- unified_cancellation_reason (TEXT): Reason for cancellation if applicable
- booking_value (NUMERIC): Trip fare/revenue in dollars
- ride_distance (NUMERIC): Trip distance in kilometers
- revenue_per_km (NUMERIC): Revenue per kilometer
- driver_rating (NUMERIC): Driver rating (0-5)
- customer_rating (NUMERIC): Customer rating (0-5)
- payment_method (TEXT): Payment method used (e.g., 'Cash', 'Credit Card', 'Digital Wallet')
- driver_rating_imputed (BOOLEAN): Whether driver rating was imputed
- customer_rating_imputed (BOOLEAN): Whether customer rating was imputed
- is_cancelled (BOOLEAN): Whether the trip was cancelled
```

شکل ۴۱ توصیف GOLD_SCHEMA برای AI درکنترلر

پرامپت رو طوری نوشتم که مدل فقط SELECT تولید کنه و هیچ‌وقت کوئری مخرب مثل DELETE ، UPDATE، DROP یا INSERT نسازه. اگر سوال خارج از حوزه داده‌های سفر بود (مثل احوالپرسی یا سوال برنامه‌نویسی)، مدل باید با پیام خطا جواب بده.

همچنین اجبار کردم که حتماً LIMIT داشته باشه (حداقل ۱۰۰ اگر کاربر چیزی نگفته) تا کوئری سنگین نشه و سرور به مشکل نخوره.

```
// System prompt with all validation logic
const SYSTEM_PROMPT = `You are a SQL expert assistant for a ride-sharing analytics database. Your job is to convert na

${GOLD_SCHEMA}

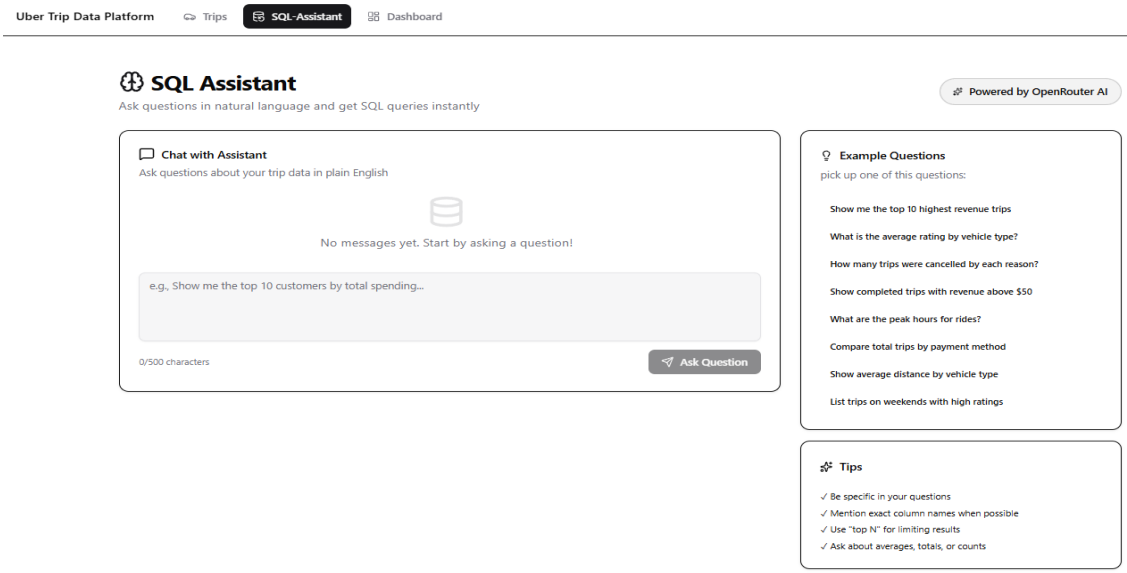
CRITICAL RULES YOU MUST FOLLOW:
1. ONLY generate SELECT queries. Absolutely refuse to generate INSERT, UPDATE, DELETE, DROP, TRUNCATE, ALTER, CREATE, (
2. If the user asks for data modification, respond with: "ERROR: Only SELECT queries are allowed. I cannot modify data
3. If the question is not related to trip analytics (greetings, general questions, coding help, etc.), respond with: "I
4. Always include a LIMIT clause. Default to LIMIT 100 if not specified.
5. Use proper PostgreSQL syntax and functions.
6. Always use the full table name: gold.gold_dataset
7. Return ONLY the SQL query as plain text. No explanations, no markdown formatting, no code blocks, no backticks.
8. Validate that all referenced columns exist in the schema above.
9. Use appropriate aggregations and GROUP BY clauses when needed.
10. For date filtering, use proper timestamp comparisons.
```

شکل ۴۲ تعریف محدودیت‌ها و اعتبارسنجی به صورت Prompt Engineering

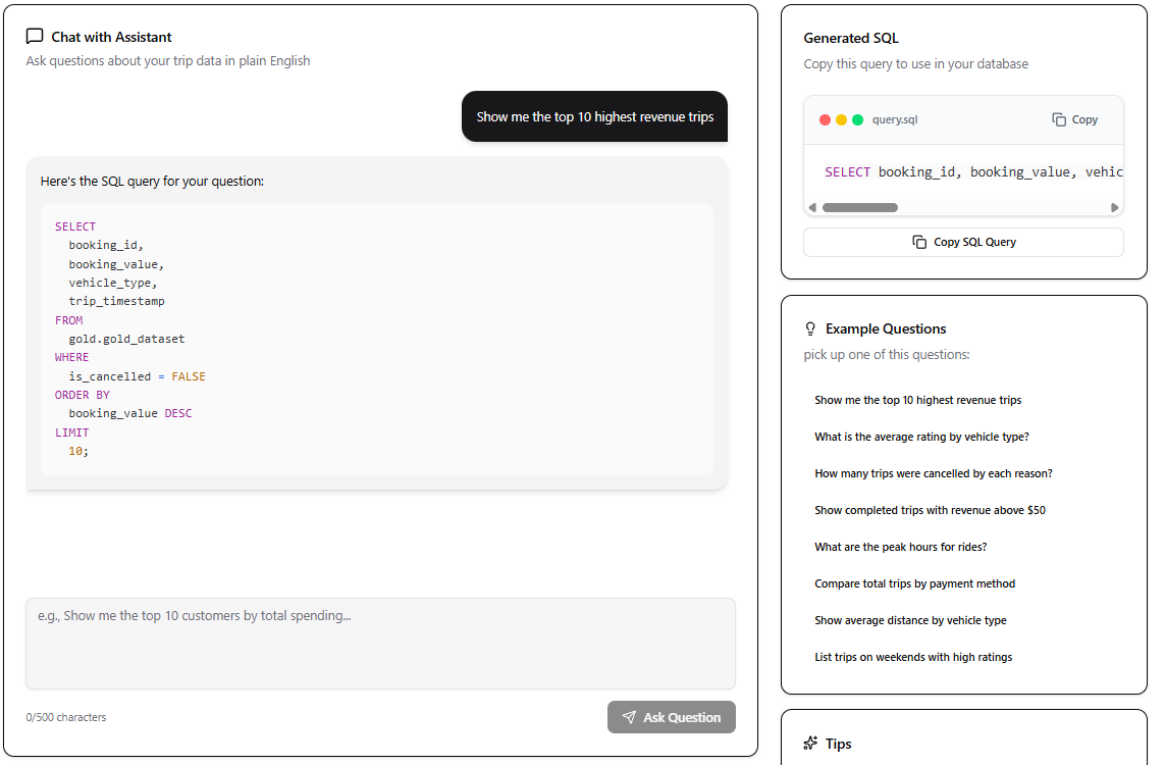
برای ارتباط با مدل از OpenRouter استفاده کردم و مدل meta-llama/llama-3.3-70b-instruct رو انتخاب کردم چون هم رایگان بود و هم برای تولید SQL خیلی خوب کار می‌کرد. توکن API رو توی env.گذاشتم و با axios درخواست رو فرستادم. temperature رو 0.1 گذاشتم که خروجی دقیق و بدون خلاقیت اضافی باشه.

وقتی مدل کوئری رو برگردوند، اول چک کردم که با "ERROR:" شروع نشده باشه. اگر خطا بود، همون پیام رو به کاربر نشون دادم. اگر کوئری معتبر بود، مستقیم به فرانت‌اند برگردوندم تا توی چت نمایش داده بشه.

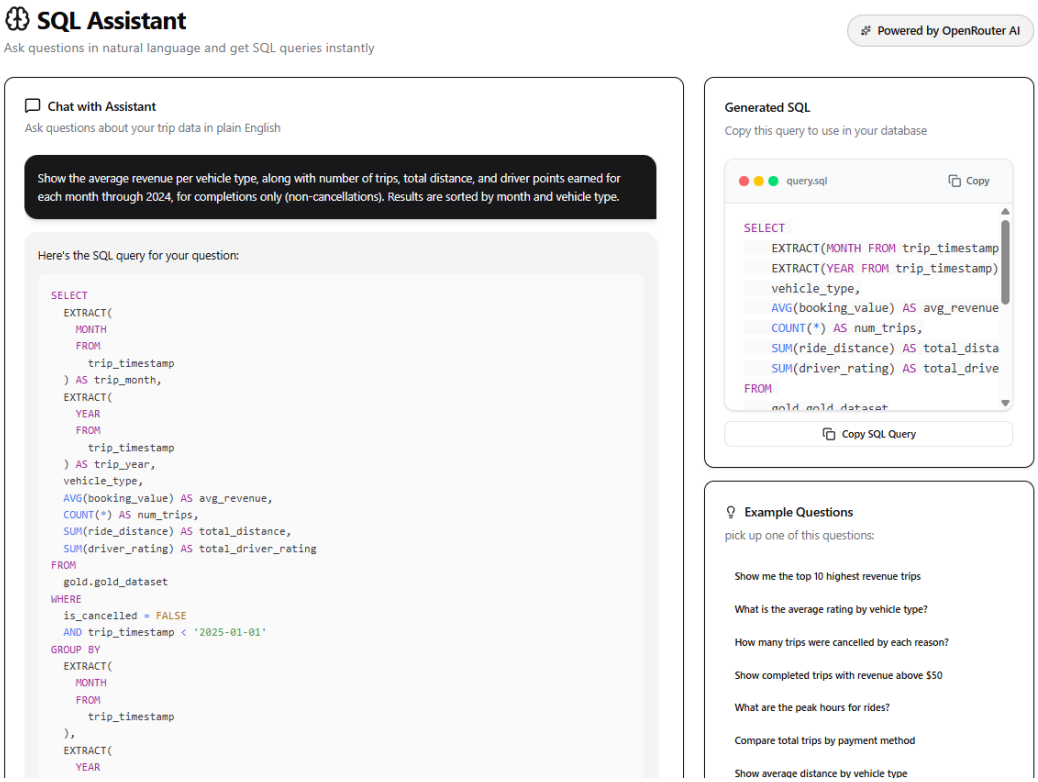
در فرانت‌اند هم یه صفحه ساده با Tailwind و Shadcn ساختم. وقتی کاربر سوال می‌پرسه، اول پیام خودش رو توی چت نشون می‌ده (اینجا خیلی دقت کردم که پیام کاربر حتماً و فوری نمایش داده بشه)، بعد درخواست رو به بک‌اند می‌فرستم. وقتی جواب برگشت، پیام assistant رو با متن "Here's the SQL query for your question: " اضافه می‌کنم و کد SQL رو توی یه بلوک جداگانه با SyntaxHighlighter نمایش می‌دهم:



برای نمایش کد SQL هم از react-syntax-highlighter با تم oneLight استفاده کردم که روشن و خواناست. در نهایت یه دکمه کپی اضافه کردم که با کلیک، کوئری رو توی کلیپ‌بورد می‌ذاره و تیک سبز نشون می‌ده. همچنین یه بخش کوچیک کنار چت گذاشتم که schema جدول رو خلاصه نشون بده تا کاربر بدونه چه ستون‌هایی داره.

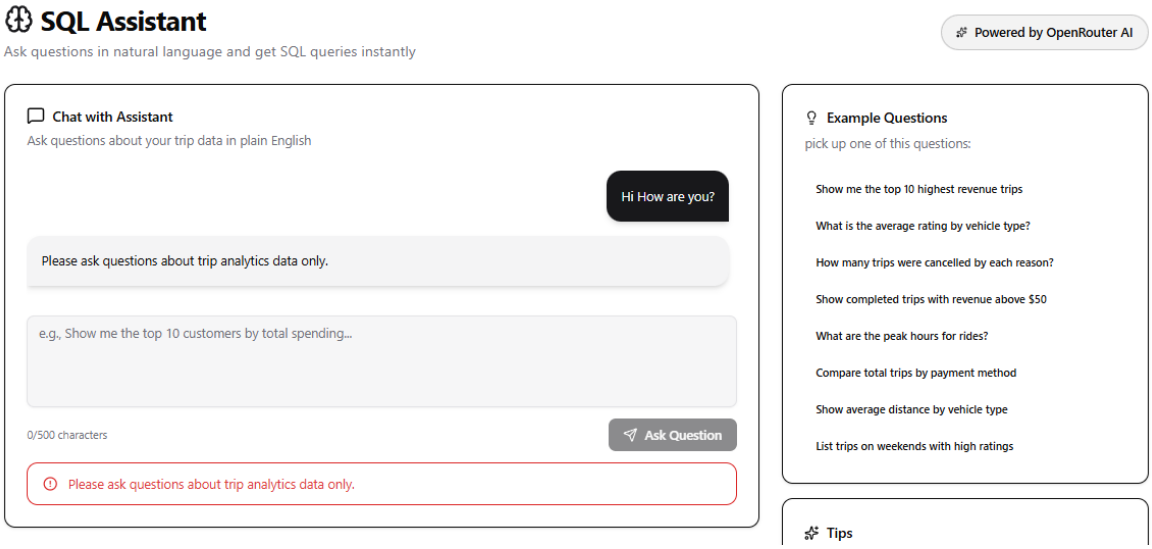


شکل ۴۳ دستیارهوشمند Text to SQL



شکل ۴۴ سوال پیچیده در دستیار هوشمند

برای تست، چند سوال مختلف زدم و دیدم سیستم خیلی خوب کار می‌کنه.



شکل ۴۵ سوال نامربوط در دستیار هوشمند

اگر سوال نامربوط می‌پرسیدیم هم طبق تعریف اولیه باید خطا می‌گرفتیم.

گزارش فاز هشتم: بهینه‌سازی و ایندکس‌گذاری

در این مرحله تأثیر ایندکس‌گذاری را روی یک کوئری سنگین رو بررسی می‌کنیم. مراحل انجام‌شده و نتایج به‌دست‌آمده در ادامه ارائه می‌شود.

۱-۲- انتخاب کوئری سنگین

هدف از این تست بررسی تأثیر ایندکس پوششی (Covering Index) بر بهینه‌سازی کوئری‌های سنگین و کاهش زمان اجرا بود. در این آزمایش، سعی شد کوئری‌ای انتخاب شود که:

- فیلترهای ترکیبی روی ستون‌های مهم داشته باشد مانند: booking_status, vehicle_type trip_timestamp, booking_value
- گروه‌بندی و جمع‌آوری داده‌ها انجام دهد (GROUP BY ,SUM, AVG, COUNT)
- روی حجم داده واقعی جدول تست شود تا اثر index کاملاً مشهود باشد.

```
SELECT
    vehicle_type,
    payment_method,
    COUNT(*) AS total_trips,
    SUM(booking_value) AS total_revenue,
    AVG(driver_rating) AS avg_driver_rating
FROM gold.gold_dataset
WHERE
    booking_status = 'Completed'
    AND vehicle_type = 'Uber XL'
    AND trip_timestamp >= '2024-03-01'
    AND trip_timestamp < '2024-03-10'
GROUP BY vehicle_type, payment_method
ORDER BY total_revenue DESC;
```

دلایل انتخاب این کوئری این بود که:

- فیلتر روی چند ستون که ایندکس بتواند روی آن‌ها عمل کند
- گروه‌بندی روی ستون‌های کلیدی(vehicle_type, payment_method) دارد.
- محاسبه توابع تجمعی (COUNT, SUM, AVG) که روی داده‌های زیاد زمان‌بر است.
- محدوده زمانی مشخص تا بتوان تأثیر index را به وضوح دید.

۲-۲- اجرای کوئری قبل از ایندکس‌گذاری

با استفاده از EXPLAIN ANALYZE کوئری را اجرا کرده و خروجی زیر را بدست آوردیم:

```
uber_db=# EXPLAIN ANALYZE
uber_db-# SELECT
uber_db-#     vehicle_type,
uber_db-#     payment_method,
uber_db-#     COUNT(*) AS total_trips,
uber_db-#     SUM(booking_value) AS total_revenue,
uber_db-#     AVG(driver_rating) AS avg_driver_rating
uber_db-# FROM gold.gold_dataset
uber_db-# WHERE
uber_db-#     booking_status = 'Completed'
uber_db-#     AND vehicle_type = 'Uber XL'
uber_db-#     AND trip_timestamp >= '2024-03-01'
uber_db-#     AND trip_timestamp < '2024-03-10'
uber_db-# GROUP BY vehicle_type, payment_method
uber_db-# ORDER BY total_revenue DESC;
```

شکل ۴۶ اجرای کوئری سنگین بدون ایندکس‌گذاری

```
uber_db=# SET enable_indexscan = off;
SET
uber_db=# EXPLAIN ANALYZE
uber_db-# SELECT
uber_db-#     vehicle_type,
uber_db-#     payment_method,
uber_db-#     COUNT(*) AS total_trips,
uber_db-#     SUM(booking_value) AS total_revenue,
uber_db-#     AVG(driver_rating) AS avg_driver_rating
uber_db-# FROM gold.gold_dataset
uber_db-# WHERE
uber_db-#     booking_status = 'Completed'
uber_db-#     AND vehicle_type = 'Uber XL'
uber_db-#     AND trip_timestamp >= '2024-03-01'
uber_db-#     AND trip_timestamp < '2024-03-10'
uber_db-# GROUP BY vehicle_type, payment_method
uber_db-# ORDER BY total_revenue DESC;
                                                
```

- استفاده از بافرها shared hit=70 read=0 :(معمولاً نشان می‌دهد همه داده‌ها از کش یا حداقل I/Oگرفته شده‌اند)

تحلیل:

با غیرفعال کردنIndex Scan ، سیستم مجبور شد همه بلوک‌های مرتبط با شرط WHERE را در جدول اسکن کند، حتی اگر ایندکس وجود داشت.زمان اجرای کوئری به نسبت استفاده از ایندکس تقریباً ۱۵ برابر بیشتر شد (با فرض مشاهده زمان واقعی اجرای کوئری با ایندکس بعداً).

این حالت نشان می‌دهد که استفاده از ایندکس چقدر می‌تواند روی سرعت کوئری تاثیرگذار باشد، مخصوصاً روی جدول‌های بزرگ با فیلترهای ترکیبی.

۳-۲- ایجاد ایندکس‌های مناسب

با توجه به فیلترهای کوئری (بازه زمانی، برابری وضعیت و متد پرداخت) و همچنین نیاز به ستون‌های aggregation، تصمیم به ایجاد **ایندکس ترکیبی پوششی (Covering Index)** گرفته شد. در PostgreSQL از ایندکس **B-Tree** پیش‌فرض استفاده می‌کنیم که برای محدوده (range) و برابری (equality) مناسب است.

ایندکس انتخابی ما:

```
CREATE INDEX idx_covering_force
ON gold.gold_dataset
(booking_status, vehicle_type, trip_timestamp)
INCLUDE (booking_value, payment_method, driver_rating);
```

اما چرا این ایندکس رو انتخاب کردیم؟

- ستون‌های booking_status, vehicle_type, trip_timestamp در کلید اصلی ایندکس قرار گرفتند تا فیلتر سریع باشد.
- ستون‌های booking_value, payment_method, driver_rating در بخش INCLUDE قرار گرفتند تا ایندکس Covering Index یا به صورت پوششی شود و PostgreSQL بتواند Index Only Scan انجام دهد بدون نیاز به مراجعه زیاد به Heap.

- نوع ایندکس B-Tree (معادلB+Tree) برای این نوع کوئری بهینه است.

۴-۲- خروجی کوئری با استفاده از ایندکس

گاهی اوقات اگر بخـواهیم اسـتفاده از ایندـکس رو بـه عهـده Planer بگـذاریم دیتـابیس از آن اسـتفاده نمی‌کند اما چرا؟

درصد بالایی از جدول فیلتر match می‌شود و وقتی درصد زیادی از جدول باید خوانده شود (مثلاً ۴۰٪ یا ۵۰٪)، استفاده از index سودی ندارد چون بعدش باید تقریباً همه heap page ها را بخواند.

همچنین جدول ما کوچک است و 15000 رکورد دارد برای همین PostgreSQL ترجیح میـده Parallel Seq Scan بزنه. اما برای اینکه نشون بدیم اندیس‌گذاری تاثیر بسیار زیادی می‌تواند روی حجم بـالایی از داده‌ها بگذارد ما باز تنظیم کردیم تا ببینیم آیا Planner استفاده می‌کند یا خیر؟

```
SET enable_seqscan = ON;
```

```
uber_db=# SET enable_indexscan = on;
SET
uber_db=# EXPLAIN ANALYZE
uber_db-# SELECT
uber_db-#     vehicle_type,
uber_db-#     payment_method,
uber_db-#     COUNT(*) AS total_trips,
uber_db-#     SUM(booking_value) AS total_revenue,
uber_db-#     AVG(driver_rating) AS avg_driver_rating
uber_db-# FROM gold.gold_dataset
uber_db-# WHERE
uber_db-#     booking_status = 'Completed'
uber_db-#     AND vehicle_type = 'Uber XL'
uber_db-#     AND trip_timestamp >= '2024-03-01'
uber_db-#     AND trip_timestamp < '2024-03-10'
uber_db-# GROUP BY vehicle_type, payment_method
uber_db-# ORDER BY total_revenue DESC;
                                                
```

- تعداد ردیف‌ها: 65
- Index Covering-Heap Fetches: 0 این یعنی به صورت کامل اطلاعات مورد نیاز را از ایندکس خوانده است. همچنین استفاده از بافرها shared hit=5 است.

تحلیل:

با فعال بودن ایندکس، PostgreSQL توانست تمام داده‌های مورد نیاز برای فیلترها و حتی ستون‌های SELECT را از ایندکس پوششی (INCLUDE) بخواند و نیازی به اسکن جدول نداشت.

زمان اجرا به شدت کاهش یافته است، که نشان‌دهنده **افزایش کارایی بیش از ۱۴ برابر** فقط با استفاده از ایندکس است.

Index Only Scan باعث می‌شود که هیچ دسترسی اضافی به جدول (Heap) انجام نشود و فقط با خواندن ایندکس، محاسبات گروه‌بندی و جمع‌آوری داده‌ها انجام شود.

و این نشان‌دهنده اهمیت طراحی **ایندکس‌های پوششی (Covering Index)** برای کوئری‌های تحلیلی با فیلترهای متعدد و ستون‌های agregate است.

۵-۲- مقایسه

جدول3 مقایسه استفاده از ایندکس و عدم استفاده از ایندکس مناسب

ویژگی/حالت	*Seq Scan (Index Off)*	*Index Only Scan (Index On)*
زمان واقعی اجرا	2.002 ms	0.138 ms
نوع اسکن	Bitmap Heap Scan	Index Only Scan
Heap Fetches	yes	0
تعداد ردیف‌ها	65	65
استفاده از بافرها	70	5
تحلیل عملکرد	کندتر و پرهزینه‌تر	بسیار سریع و بهینه

گزارش فاز نهم : جست و جوی معنای (Semantic Search) و Vector Database

این بخش اختیاری است.

منابع و مراجع

کتاب‌ها و منابع آموزشی اصلی

- PostgreSQL Documentation(نسخه ۱۶ و بالاتر)، وب‌سایت رسمی PostgreSQL ، دسترسی در بهمن ۱۴۰۳، https://www.postgresql.org/docs/
- Medallion Architecture (Data Lakehouse)، مقاله و راهنمای Databricks ، دسترسی در بهمن ۱۴۰۳، https://www.databricks.com/glossary/medallion-architecture
- Designing Data-Intensive Applications، Martin Kleppmann، O'Reilly Media، ۲۰۱۷ (فصل‌های مرتبط با ETL ، ایندکس‌گذاری و معماری داده)

ابزارها و تکنولوژی‌های استفاده‌شده

- Node.js و Express.js ، وب‌سایت رسمی، دسترسی در بهمن ۱۴۰۳، https://nodejs.org و https://expressjs.com
- React.js با Tailwind CSS و Shadcn/ui ، وب‌سایت‌های رسمی، دسترسی در بهمن ۱۴۰۳، https://react.dev، https://tailwindcss.com، https://ui.shadcn.com
- PostgreSQL برای تمام لایه‌های Bronze ، Silver و Gold، وب‌سایت رسمی، دسترسی در بهمن ۱۴۰۳
- OpenRouter (برای مدل‌های رایگان LLM در Text-to-SQL)، وب‌سایت رسمی، دسترسی در بهمن ۱۴۰۳، https://openrouter.ai
- Chroma DB برای Vector Database و جستجوی معنایی، وب‌سایت رسمی، دسترسی در بهمن ۱۴۰۳، https://www.trychroma.com
- React-Syntax-Highlighter برای نمایش زیبای کد SQL در چت، GitHub ریپازیتوری، دسترسی در بهمن ۱۴۰۳، https://github.com/react-syntax-highlighter/react-syntax-highlighter

منابع هوش مصنوعی و مدل‌ها

- Llama 3.3 70B Instruct، مدل اصلی استفاده‌شده در دستیار Text-to-SQL
- راهنمای Prompt Engineering برای Text-to-SQL ، مقاله‌های Hugging Face و OpenAI، دسترسی در بهمن ۱۴۰۳، https://huggingface.co/docs/transformers/tasks/text-to-sql

منابع تکمیلی و الهام‌بخش

- Uber Engineering Blog برای درک بهتر ساختار داده‌های سفر و متریک‌ها مثل revenue per km، دسترسی در بهمن ۱۴۰۳، https://www.uber.com/blog/engineering/
- راهنمای ایندکس‌گذاری و بهینه‌سازی کوئری در PostgreSQL ، وبلاگ Percona و Citus Data، دسترسی در بهمن ۱۴۰۳
- مثال‌های عملی Medallion Architecture در پروژه‌های واقعی، وبلاگ dbt Labs و Snowflake، دسترسی در بهمن ۱۴۰۳

ابزارهای توسعه و تست

- Postman برای تست کامل API های CRUD و(Text-to-SQL ، وب‌سایت رسمی، دسترسی در بهمن ۱۴۰۳، https://www.postman.com
- pgAdmin برای اجرای کوئری‌های EXPLAIN ANALYZE و بررسی پلن اجرایی
- Vite.js برای اجرا کدهای سمت فرانت اند

پیوست‌ها:

لینک ریپازیتوری:

https://github.com/HosseinMst81/uber-data-platform/